pii · personal information index

User and Administration Manual

Marcelo Correa

Version 0.2.0 · build 2026-11-04 10:27 UTC · commit `4249d5b2fa85`

Setup, daily usage and the technical reference

Contents

Contents ii

1 What pii is 1
1.1 The three things it does . . . 1
1.2 The shape of the system . . . 1
1.3 A graph is data . . . 2
1.4 The journal, and the verdicts . . . 4
1.5 What it remembers, and where . . . 5
1.6 Capabilities are a ceiling . . . 6
1.7 How a run is named, and what a trace looks like . . . 7
1.8 What pii refuses to claim . . . 7
1.9 What it deliberately is not . . . 8
1.10 How to read this manual . . . 8

2 Installation 10
2.1 What you need . . . 10
2.2 The binary and the state directory . . . 10
2.3 The installer, stage by stage . . . 11
2.4 Environment and scope . . . 12
2.5 The configuration file . . . 13
2.6 Credentials . . . 14
2.7 Supervising the daemon . . . 15
2.8 The graphs an installation serves . . . 16
2.9 Verifying the installation . . . 16
2.10 Rolling an installation back . . . 17
2.11 Failure modes of the install itself . . . 18

3 Daily use 19
3.1 Asking the daemon a question . . . 19
3.2 One-shot prompts . . . 20
3.3 Running a graph . . . 20
3.4 The queue . . . 21
3.5 Reading a run . . . 22
3.6 Stores, memory and the indices . . . 24
3.7 A cadence that works . . . 25
3.8 What a quiet day looks like . . . 25

4 Operating 26
4.1 Backup and restore . . . 26
4.2 Retention . . . 27
4.3 What retention actually removes . . . 28
4.4 Compaction is not retention . . . 28
4.5 The restore drill . . . 29
4.6 Upgrading . . . 29
4.7 Logs, audit, and the death that leaves no line . . . 30
4.8 Disk is owned, bounded and visible . . . 31
4.9 When the daemon is wedged . . . 32
4.10 Failure modes that have actually happened . . . 32
4.11 The first five minutes of an incident . . . 33
4.12 Two rules that keep a host boring . . . 34

5 Command reference 35
5.1 The shape of the surface . . . 35

5.2 Starting work . . . 37
5.3 The queue . . . 41
5.4 Stores, backups and memory . . . 42
5.5 Knowledge, skills and indices . . . 44
5.6 Observation and decisions . . . 45
5.7 Devices, sandboxes and scaffolding . . . 46
5.8 Exit codes . . . 46

6 Configuration **48**
6.1 Two files, one effective config . . . 48
6.2 The project layer, and what it may not say . . . 48
6.3 The file is strict . . . 49
6.4 The sections . . . 49
6.5 Where the files are . . . 51
6.6 Secrets are not configuration . . . 52
6.7 Environment variables . . . 53
6.8 Changing a setting . . . 53
6.9 A production file, in full . . . 54
6.10 Refusals, and what they look like . . . 55

7 Observability **56**
7.1 Four surfaces, four questions . . . 56
7.2 The anatomy of one event . . . 57
7.3 The name registry . . . 57
7.4 Metrics . . . 58
7.5 Health . . . 60
7.6 The run journal . . . 61
7.7 The audit trail . . . 62
7.8 Incidents and the loop . . . 63
7.9 Correlating a failure . . . 63

8 The graph library **65**
8.1 What a graph is, as an operator . . . 65
8.2 The inventory . . . 65
8.3 The lint status of the library . . . 66
8.4 How the pieces compose . . . 67
8.5 The operator flows . . . 68
8.6 The coder samples . . . 71
8.7 The awareness graphs . . . 72
8.8 Preflight and the fragment . . . 73
8.9 The starter graphs . . . 73
8.10 What the graphs can do to the machine . . . 74
8.11 Reading a graph's failure . . . 75

9 Memory **76**
9.1 The four things a document carries . . . 76
9.2 The three layers, and what each is for . . . 77
9.3 How a memory is found: three lanes, one ranked list . . . 77
9.4 The graph lane, and why it often contributes nothing . . . 78
9.5 Consolidation: when many atoms become one scene . . . 79
9.6 The commands, and what each one touches . . . 80
9.7 What goes wrong, in the order you will meet it . . . 80
9.8 How memory is bounded . . . 81

Where to look next **82**

What pii is 1

pii is a *personal information index*: one Rust binary that keeps stores of runs, memory, entities and lessons on disk, executes *graphs* — small programs written as data — on a daemon, and records every state change in a journal that can be replayed, counted and audited. This chapter answers three questions before the rest of the manual asks anything of you: what the program does, what it is made of, and which of its answers you are allowed to trust. Every number and path in this chapter was read from the repository at the revision the manual was built from, not from a design document; where a mechanism exists because something failed, the failure is named.

It is not a chat application that happens to have a database. It is an execution engine whose every action leaves evidence, and the questions you will ask of it are of the form "what ran, what did it decide, and can I prove it". The design follows from that: if a fact cannot be shown from the journal or from a store, pii treats it as unknown rather than as true. The three exit codes of `pii health` are the smallest example — a daemon that answers from a different build than your client is reported as drift, not as health.

1.1 The three things it does

1. **Runs work.** A graph is a set of nodes — ask a model, run a command, judge a set of exit criteria, call another graph — connected by named ports. pii executes it, journals each transition, and returns a verdict rather than a transcript.
2. **Remembers what it learned.** Runs, node-call events, memory documents, entities and lessons live in redb stores with counters, snapshots and, since the retention row, an age-based policy that bounds them.
3. **Tells the truth about itself.** `pii health` answers from a record kept current by the write path rather than by replaying history at read time, and the answer names the build, the config file's hash, each store's path and format version, and the mounts the sandbox actually granted.

Those three are one mechanism seen from three sides. The journal is what makes a verdict provable; the stores are what make a second run cheaper than the first; the identity payload is what makes an upgrade, a rollback or a wrong state directory visible instead of merely suspected.

1.2 The shape of the system

The workspace is twenty-two `pii-*` crates plus `xtask`, the internal tool that runs the quality gate. Four binaries come out of it: `pii` (the daemon and the command line in one file), `pii-mcp` (a Model Context Protocol server), `pii-web` (a small HTTP surface) and `pii-evolve`. Only `pii` is required to operate the product; the installer places the other three beside it, and `pii-mcp` is staged as `pii-mcp.new` rather than swapped

in, because its callers are other programs whose behaviour has to be measured before the swap.

Binary	Crate	What it is for
`pii`	`pii-cli`	The command line, and the daemon when invoked as `pii daemon`. Embeds the engine in-process for a one-shot prompt.
`pii-mcp`	`pii-mcp`	A Model Context Protocol server over the same engine, for tools that speak MCP. Installed as `.new`; the swap is an operator step.
`pii-web`	`pii-web`	A separate HTTP binary, not linked into `pii`.
`pii-evolve`	`pii-evolve`	A separate evolution binary, not linked into `pii`.

Inside `pii` the layering is strict, and it is the reason an operator can reason about failure at all. The control commands — `run`, `queue`, `health`, `memory`, `skill` — never open a store: they dial `control.sock`. The daemon owns the stores, the run pool, the timer wheel and the channels. The engine executes one run at a time per worker and journals every node transition. `pii-store` is the only crate that knows redb exists, and the commands that do open a file (`store`, `backup`, `restore`, and the offline `graph run` and `graph trace` with `-db`) take redb's exclusive lock and refuse while a daemon holds it.

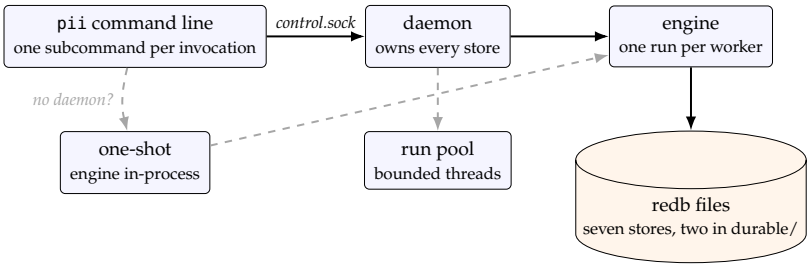


Figure 1.1: Two ways in and one owner. The command line either reaches the daemon over its socket, or — for a one-shot prompt only — builds the engine in its own process. Notice that the store has exactly one writer, which is what makes an offline command refuse rather than contend.

The distinction matters in practice. `pii store compact` and `pii backup` are two of the commands that do open the files, and both refuse while a daemon holds them instead of copying a store mid-write. Everything else — `run`, `queue`, `health`, `memory`, `skill` — is a protocol call.

1.3 A graph is data

A graph file is TOML with five top-level facts: a `name` (the key `pii run start` resolves), an `entry` node, an optional `exit` node, a `caps` list, and an optional `[limits]` table. Nodes and edges follow. This is the beginning

of the development queue's own graph, at the revision this manual describes:

```
name = "queue"
entry = "select"
exit = "record"
caps = ["spawn", "proc", "fs_read", "fs_write"]

[limits]
max_wall_ms = 6000000

[[node]]
id = "select"
kind = "select"

[[node]]
id = "implement"
kind = "call"
graph = "spec"
sha256 = "771d098b39db7369dd71ed1fb23ffa63b90b25fed7788c830e4bfb214c424443
    ↪ "
caps = ["spawn", "proc", "fs_read", "fs_write"]

[[edge]]
from = "select"
port = "ok"
to = "implement"
kind = "data"
```

Three things in those lines carry most of the design. The `sha256` on a `call` is a pin: the callee is spliced into the host at load time, so a change to the callee invalidates the pin until someone re-pins it deliberately. The `caps` on the call node are the host's promise that it holds what the callee needs. And the `port` on an edge is a name, not a boolean — `select` leaves by `ok`, `empty` or `blocked`, and each can go somewhere different.

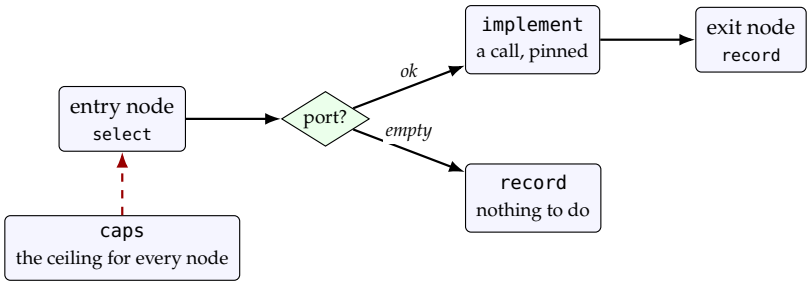


Figure 1.2: A graph from the operator's side: one entry, named ports, and an exit that makes the graph usable as a part of another graph. A graph without an exit is a run, not a component — nothing can call it.

The node set is small by design. Seven kinds live in `pii-graph` and need no services at all: `transform`, `route`, `emit`, `wait`, `timer`, `refuse` and `require`. The daemon registers the rest, because they need a store, a socket or a process.

Kind	What it does, and what it costs
`transform`	Rewrites the value on the port. No process, no model.
`route`	Chooses a port from the value it received. The cheapest decision in the system.
`wait`, `timer`	Sleeps, or waits for a wall-clock moment. `timer` is what makes a graph schedulable.

Kind	What it does, and what it costs
`refuse`, `require`	Ends the run with a named refusal, or refuses to start unless a condition holds. A refusal is a verdict, not an exception.
`coder`	Spawns a coding agent (`pi` or `claude`) and takes the agent's verdict from the run of its criteria, never from the agent's own report.
`judge`	Runs a spec's exit criteria through a shell and reports `ok`, `fail` or `unrunnable`.
`select`, `record`	Claim one work item from the queue store; write a verdict back to it.
`agent`, `tool`, `worker`	Call a model, a built-in tool, or a supervised child process. All three need `spawn` or `proc`.
`gate`	Suspends the run for an approval, and resumes it on the decision.
`post`, `mqtt`	Send a message to a configured channel, or publish to a broker.
`memory`, `remember`, `recall`	Consolidate the memory store; write a fact from a run; read it back in a later run.
`heal_detect`, `heal_record`	Run a gate, record the failures it found, and keep the record.
`spawn`	Starts a child run on the same engine.
`call`	Splices another graph in at load time. Not a runtime kind: it becomes the callee's nodes before the run starts.

A graph that lints can still be wrong. A graph that does not lint will not run: `pii graph lint` is the only check that refuses an edge to a port the node does not declare, a capability the graph does not hold, or a pin that no longer matches the callee's file.

1.4 The journal, and the verdicts

Every run writes an append-only journal, and the events are a fixed vocabulary of seventeen: `Started`, `TickStarted`, `NodeStarted`, `NodeFinished` (with its port and the hash of the value), `NodeFailed`, `NodeProgress`, `NodeCancelled`, `NodeDead`, `NodeInterrupted`, `Suspended`, `Resumed`, `NodeObserved`, `Finished`, `EffectStarted`, `EffectConfirmed`, `NodeRetried` and `RunSpawned`. That vocabulary is the reason a trace can be read rather than guessed at. Two of the seventeen exist only for effects: an effect node journals its idempotency key *before* it acts, so a key with no confirmation means the effect may or may not have happened, and a replay says so instead of acting twice. A run's stored state is one of five:

Status	Meaning
`Running`	Executing, or waiting to be resumed.
`Suspended`	Stopped at a resume key — an approval, a timer — and resumable.
`Succeeded`	Reached its terminal node and reported success.
`Failed`	Reached a terminal node and reported failure, or was cancelled.
`Refused`	An agent call was refused before any spawn. It would not start, which is not the same as failing.

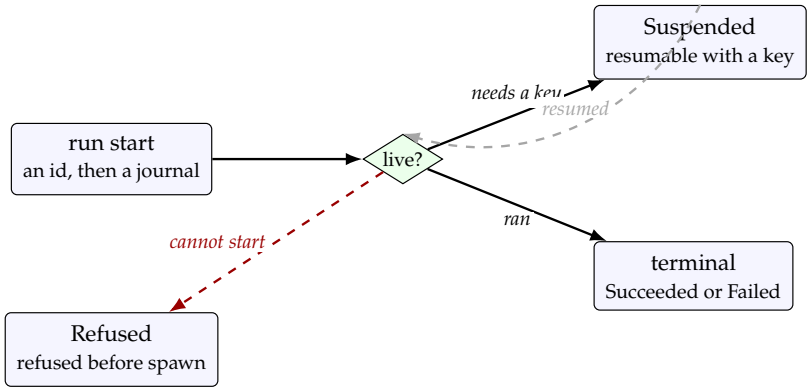


Figure 1.3: Five statuses, and the loop that matters: a suspended run is not a finished one. Retention keeps a Running or Suspended run however old it is, because the daemon may still resume it.

A direct run reports one of three terminals, and `pii graph run` maps them to exit codes an operator can branch on: success is 0, a failure is 1, a suspended run is 4, and a graph the validator refuses is 3 — before anything is executed and, deliberately, before any store file is created.

1.5 What it remembers, and where

Seven store files appear in `pii health`. Six of them are what a backup copies: the main store, memory, approvals, heal, channel and timer. The queue has its own file and is reported alongside them, but it is not in the backup set at this revision — a detail that matters the first time you restore and find the work list empty.

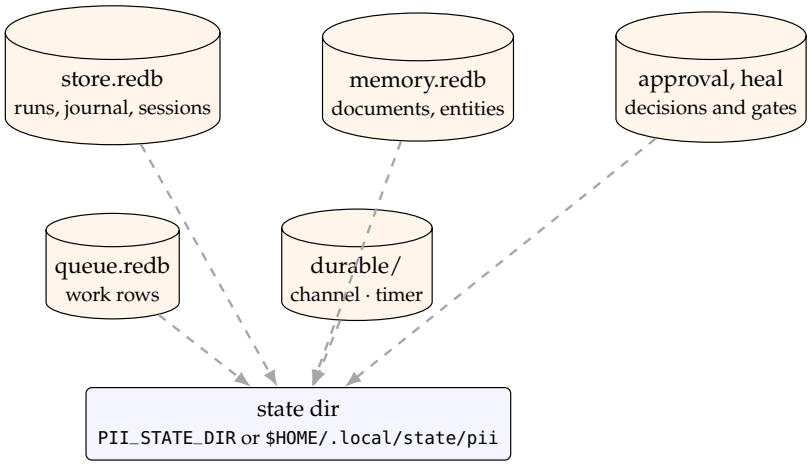


Figure 1.4: Every store except the socket lives under one state directory. The durable pair sits in `durable/` because the runtime directory is a tmpfs on a normal host, and a channel cursor or a timer that vanishes at reboot is worse than no timer at all.

The main store is a redb file, which never shrinks on its own: space is returned only at the end of the file, and only by compaction. That is the reason a retention policy exists at all, and the reason `pii store compact` is documented as narrow — it reclaims free pages inside the file, not live history.

1.6 Capabilities are a ceiling

Twelve capability bits exist, and every one of them is checked twice: the graph declares what it may hold, and each node may hold only what its graph holds. A node that asks for more is refused at load, not degraded at runtime.

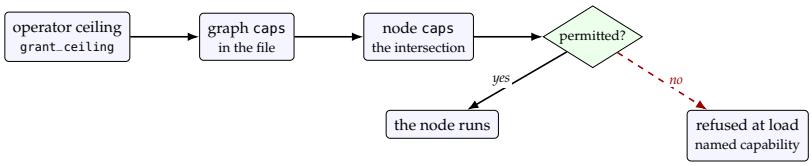


Figure 1.5: The ladder narrows downwards and never widens. A `call` can grant only what its host grants, which is why `night` holds what `queue` needs before it may call it.

Capability	What holding it allows
`fs_read`	Read files inside an allow path scope.
`fs_write`	Write files inside an allow path scope.
`proc`	Run commands — this is what lets a judge execute exit criteria.
`spawn`	Start an agent or a child process. Every model call needs it.
`net`	Reach the network, subject to the config's host grants.
`hw`	Touch hardware through the HAL.
`secrets`	Read secret material.

Capability	What holding it allows
`memory_read`, `memory_write`	Read from, or write into, the agent's memory.
`post`	Send a message into a configured channel.
`mqtt_read`, `mqtt_write`	List and subscribe to a broker's devices; command one through its allowlist.

The measured consequence is worth stating plainly: a coder inside a graph whose `caps` omitted `fs_write` did not silently write nothing. It stopped and reported "no write or execute tool is available". That sentence is the feature working; a degraded coder that pretends to work is the failure it replaced.

1.7 How a run is named, and what a trace looks like

Run identifiers are allocated by the store, not by the caller, so two processes cannot collide: the daemon's own logs and the graph files quote them in the form `r000000000130`. A caller may pass `-run-id` on the offline path, which is how the test suites keep a fixture's journal findable.

`pii health` is the widest single view of what a process is. Its text form is short enough to quote in a bug report, and every line is a fact a command can be held to:

```
pii <version> (<revision>, <target>)
client: <revision>
config: <path> sha256=<hash>
stores:
  main: <path> (<bytes> bytes, format_version=<n>)
  memory: <path> (<bytes> bytes, format_version=<n>)
mounts:
  workspace [rw]: <root>  (<policy>)
since_deploy:
  starts: <n>
counts:
  runs: <n>
```

The field names, their order and their punctuation are exact; the values are whatever your daemon reports. Read it as four claims, in order. The first line is *what the daemon is*: version, revision and target. The second is *what your client is*: if the two revisions differ, `pii health` exits 3 and prints a line beginning `DRIFT`, and that is the correct reading immediately after installing a new binary and before restarting the service. The `config` line is the hash of the file the daemon actually loaded — not the path you think it loaded. The `stores` block is the one that catches the most common installation mistake: a daemon whose store list is empty is a daemon pointed at an empty directory, and it will happily report itself active.

1.8 What pii refuses to claim

Three refusals are part of the design, and an administrator meets all three.

- A store whose format version is newer than the binary supports is not read half-way. The daemon exits 2 and writes nothing to the file, with the version and the path in the message.
- A health answer whose build revision is unknown on either side is reported as unverified, with a non-zero exit code, rather than as agreement. Unknown is never agreement.
- A retention sweep will not remove a row it cannot date. A run created before the clock was recorded, or a fixture with no timestamp, is kept and counted, because deleting on a missing timestamp destroys exactly the runs an operator is looking for.

1.9 What it deliberately is not

Four absences are decisions rather than gaps, and each one shapes how you operate the thing.

- **Not a cluster.** The stores are redb files with an exclusive lock, and the daemon claims the durable pair before it binds its socket. A second daemon pointed at the same state directory is refused, by name, at start. Scaling out means changing the store, not adding a replica.
- **Not an online compactor.** `pii store compact` runs redb's in-place compaction, and it needs the daemon stopped, because it takes the same exclusive lock the daemon holds for its lifetime. There is no background compaction to tune or to fear.
- **Not a cron daemon.** A `[[cron]]` entry carries a `schedule` string, but that string is a display mirror: what fires a graph is a `timer` node at its entry. A graph whose entry is not a timer will not be started by the schedule, and the config will look correct while nothing runs.
- **Not a TLS client for MQTT.** The broker client speaks plain TCP. A remote broker with credentials needs an explicit acknowledgement in the config, and the honest answer for a non-loopback broker is a tunnel.

Knowing these four saves the two most expensive mistakes in operation: pointing a second process at a live state directory, and believing a schedule that only mirrors itself.

1.10 How to read this manual

Chapter 2 gets a host running: the two directories, the installer, the unit, the credentials, and the three commands that prove the result. Chapter 3 is what a normal day looks like — health, one-shot prompts, graphs by name and by file, the queue, and how to read a trace. Chapter 4 is the part nobody reads until something breaks: backup and restore, retention, upgrades, logs, disk ownership, the deploy card and the failure modes that have actually happened.

Chapters 5 and 6 are reference material and are meant to be searched, not read. Chapter 7 is the metrics and event surface, and Chapter 8 is the graph library: which graphs an operator runs and what each will do to the machine.

Two conventions run through the book. A path in `typewriter` is literal and was copied from the repository or from a command's own output.

A number in the text is a measurement recorded in the repository, with the row or document that recorded it named where the number is load-bearing. Where this manual and a design document disagree, the manual is describing the code at the built revision, and the disagreement is itself worth reporting.

Installation 2

pii installs as a *user* service by default. There is no package and, in that scope, no root-owned state: the binary, its configuration, its graphs and its stores all live under your own home, which is also what makes an upgrade a matter of replacing files that the installer has already moved aside. A system scope exists — `SCOPE=system`, with `/etc/pii` and `/var/lib/pii` — and everything below is written to be read with either in mind. This chapter is the whole install path — the two directories, the script that fills them, the unit that supervises the process, the credential file, and the commands that prove the result. It also lists the preconditions the installer checks before it changes anything, because every one of them was added after an install that half-succeeded.

2.1 What you need

- a Linux host with a writable `$HOME` and a user session. systemd `-user` is how the daemon is normally supervised, but it is not required: the daemon is a foreground process that can be started by hand, and on a host without a user bus that is the only way it runs;
- a kernel that can enforce the sandbox profile. Landlock is the mechanism; if the profile cannot be enforced the daemon *refuses to start* with exit code 4 rather than falling back to unconfined;
- the `pii` binary, built for the host's architecture, plus the three sibling binaries if you want them;
- `curl`, which the installer uses for one precondition probe;
- Python 3.11 or newer with `tomllib`, which the installer uses to merge and verify the configuration file. The script locates it, or falls back to a `uv` provisioned interpreter under `_tmp/`;
- a provider credential for any workflow that calls a model. It is kept in the environment of the service, never in a file that is committed;
- a *deploy card* for the revision you are about to install. It is a JSON file under `_tmp/deploy/` named `card-<rev>.json`, keyed by the seven- or twelve-character short revision of `HEAD`. The installer refuses without one.

The deploy card is the one precondition that surprises people. It is not bureaucracy around the install script: it is the record that a specific revision was proved red on its base, green on its branch, gated, and rehearsed for rollback. A card is rendered, approved or refused with `pii deploy-card`, and a card missing any of its five fields is refused with the field named and no approval path printed.

2.2 The binary and the state directory

The binary is self-contained. Its *state* lives in a directory chosen once, by one function, in this order:

1. `PII_STATE_DIR`, when set and non-empty;
2. otherwise `$HOME/.local/state/pii`.

That single resolver is deliberate. Before it, several commands each had their own notion of where the store lived, and a command could read a store the daemon was not writing. The measured symptom was not a crash: it was a CLI reporting an empty run list while the daemon served hundreds of runs from another file. If you set `PII_STATE_DIR` for a test, set it for the daemon too, or you are looking at two different systems.

The *runtime* directory is a second, separate rule, and mixing the two up is the most common installation defect:

1. `PII_RUNTIME_DIR`, when set;
2. `[core].runtime_dir` from the config file;
3. `/run/user/<uid>/pii`.

The runtime directory holds the control socket and, on a normal host, is a tmpfs. That is why the durable stores — the channel cursor and the timer wheel — default to `<state dir>/durable` and never to the runtime directory: a timer that disappears at reboot is worse than no timer at all. It is also why the installer symlinks the graphs directory into the runtime directory on every start rather than copying it there.

The shape of an installation

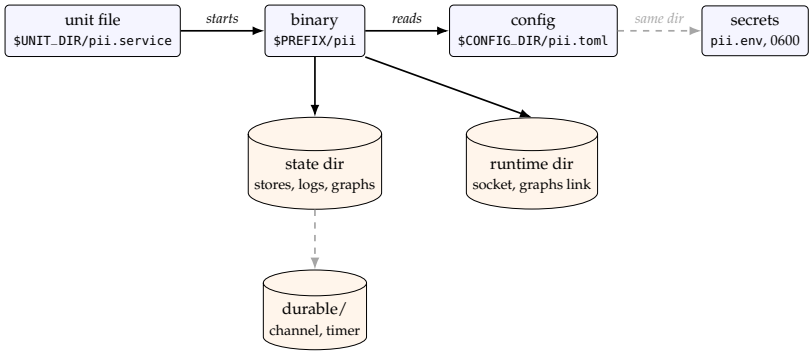


Figure 2.1: What an installation owns, in the user scope. Notice the two dashed edges: the key file sits beside the config but is read by the service, and the unit is only a file until an operator enables it.

2.3 The installer, stage by stage

One command does the whole install, and it is idempotent. It never deletes: anything it replaces is moved into a dated directory first, so a failed install leaves you able to compare and to roll back.

```
bash deploy/install-pii.sh                    # SCOPE=user (the default)
SCOPE=system bash deploy/install-pii.sh       # /etc/systemd/system, /etc/
    ↪ pii, /var/lib/pii
CONFIG_ONLY=1 bash deploy/install-pii.sh      # rewrite the config and
    ↪ credentials only
```

The stages run in a fixed order, and the preconditions are all checked before the first file moves. That ordering is not cosmetic: it means an aborted install changes nothing, which is the only property that makes a

script safe to run on a host you care about. The one exception is `CONFIG_ONLY=1`, which skips the build, the card, the binaries and the unit, and touches only the config file and the credential file.

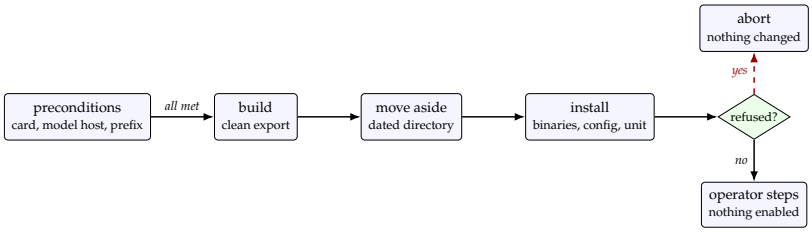


Figure 2.2: The installer's shape. Every refusal happens either before the build or in the config-merge step, which restores the replaced file byte for byte. The script ends by printing what an operator still has to do; it never enables or starts the unit.

What the preconditions cost when they fail

Each precondition exists because the failure it prevents was measured.

- **The model host must answer.** The script does `GET <llama host>/models` with a five-second limit and aborts if it does not. Without this probe an install succeeds and every graph that needs a model fails at spawn, minutes later, with the daemon looking healthy.
- **The named graphs must exist.** The generated configuration names `discord-triage` and `discord-digest`; the script refuses if either file is missing from `graphs/`, because a config that names a graph the daemon cannot load is a daemon that starts with a channel and no work.
- **The prefix must be writable.** It is created if absent and refused if it cannot be written — before the build, so a permissions problem costs seconds rather than a release compile.
- **The working tree must be a git checkout.** The card is keyed by the short revision of `HEAD`, and there is no install without a card.

The build itself is worth one paragraph, because two deployment defects are recorded in its comments. The script exports `git archive HEAD` into a temporary directory and builds there. Measured on three consecutive deploys, building in the working tree shipped binaries that reported `<rev>-dirty` whenever any path was uncommitted — including a worker's leftover — so the revision the deploy card quotes carried a suffix meaning *this is not the commit you verified*. The export has no `.git`, so `build.rs` cannot read the revision from it; the script therefore passes `PII_BUILD_REVISION` explicitly. That was measured too: the first export-built binary reported `unknown` for a build correctly made from a known commit, and `pii health` classified it as unverified for ever after.

2.4 Environment and scope

Every path the installer writes is an environment variable with a default, so a second installation on the same host — a test prefix, a different user — needs no editing of the script.

Variable	User scope default	System scope default
SCOPE	user	system
PREFIX	$HOME/.local/bin	$HOME/.local/bin (the static unit in deploy/ instead names /usr/local/bin/pii)
CONFIG_DIR	$HOME/.config/pii	/etc/pii
STATE_DIR	$HOME/.local/state/pii	/var/lib/pii
RUNTIME_DIR	/run/user/<uid>/pii	/run/pii
UNIT_DIR	$HOME/.config/systemd/user	/etc/systemd/system
BIN_SRC	<repo>/target/release	same
LLAMA_HOST	http://10.10.10.217:8082/v1	same

Three more variables change what the script does rather than where it writes. SKIP_BUILD=1 installs the binaries already in BIN_SRC and is how a host without a Rust toolchain takes a release; CONFIG_ONLY=1 rewrites only the configuration and the channel credentials, with no build, no card, no binaries and no unit, which is the supported way to change a model host or a token on a running installation; and PII_PYTHON pins the interpreter used for the config merge.

2.5 The configuration file

Behaviour lives in one TOML file, and the file is strict: an unknown key is a load error, so a typo in a sandbox or a runtime path cannot silently become a default. The installer writes it by *merging* rather than regenerating, and then verifies that every table and key path of the replaced file is present in the candidate, naming each miss instead of proceeding. Before that change, a re-install rewrote the file from the template and carried only model forward — which is how an operator's sections disappeared.

The sections the daemon reads are these, and nothing else:

Section	What it decides
`grant_ceiling` (top level)	the capability names no graph or node may request beyond; empty means no ceiling
`[core]`	model, runtime dir, durable dir, log file and rotation, trajectory root and its bounds, the measurement switches
`[llama]`	the base URL a `llamacpp-server:` model dials
`[net]`	egress host patterns and whether private destinations are allowed
`[memory]`, `[memory_-graph]`	the memory store's path and graph settings
`[[worker]]`	supervised child processes, with their own in-flight limits
`[[channel]]`	inbound message sources and the graph each message runs
`[[mqtt]]`	broker connections, topics and the graph per message
`[[cron]]`	named schedules, and the graph each one names
`[hal]`, `[wasm]`, `[evolve]`	optional subsystems; absent means off
`[[model_-tier]]`	named model tiers

The `[core]` keys an administrator actually touches are `model`, `runtime_-dir`, `durable_dir`, `log_file`, `log_rotate_bytes` (default 10485760), `log_keep_files` (default 5), `trajectory_dir` (absent means trajectories are off), `trajectory_max_bytes` (default 512 MiB), `trajectory_max_-files` (default 50 000), and the two measurement switches `observer` and `audit`, which default to `on` and must stay on in production.

The path the CLI reads is not always the path the service reads

`pii daemon` takes `-config`, defaulting to `$PII_CONFIG` and then `$HOME/.pii/pii.toml`. The installer writes `$CONFIG_DIR/pii.toml` — `$HOME/.config/pii/pii.toml` in the user scope — and puts `PII_CONFIG` in the unit. The consequence is exact and worth memorising: a shell that runs `pii` without `PII_CONFIG` set is reading a different file from the one the daemon loaded, and `pii health` will show you the daemon's file by path and hash if you ask. Export `PII_CONFIG` in your shell, or accept that the two are separate.

The same shape applies to the store. `pii daemon -help` documents the default as `<runtime dir>/store.redb`, while the store commands and the backup default to the state directory; the change that would make the daemon agree had not landed at this revision. Do what the shipped unit does: pass `-store` explicitly, pointing at `$STATE_DIR/store.redb`.

2.6 Credentials

Model providers and channels are reached with values taken from the environment of the process that needs them. Nothing in the configuration file is a secret, and no pii command prints one: provider errors and events are scrubbed of the key before they are written.

The convention in this deployment is a single environment file next to the config, owned by the user, mode `0600`, read by the service unit and never by the repository:

```
install -m 600 /dev/null ~/.config/pii/pii.env
# then edit it to hold the provider key(s) and the channel ids, and
    ↪ nothing else
chmod 600 ~/.config/pii/pii.env
```

The unit reference is written with a leading dash — `EnvironmentFile=-$CONFIG_DIR/pii.env` — which makes the file optional. The difference matters: without the dash, a missing file is a unit that fails to start, and the error is about systemd rather than about the credential. With it, the daemon starts and reports what it does not have — `channels: []` — which is a state to report, not a failure to diagnose.

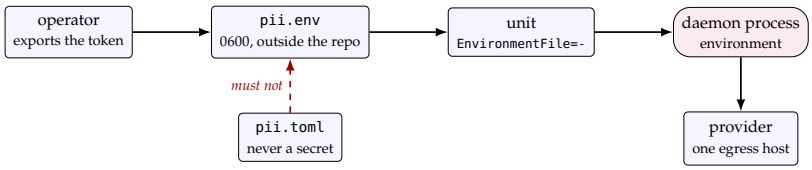


Figure 2.3: The secret path. The value reaches the daemon's environment and nothing else; the config file names the shape of the connection, never the credential. A key pasted into a chat or a commit is compromised, and rotation is the only remedy.

Named variables, so a grep for one of them in the tree is a finding: `OPENAI_API_KEY` and `OPENAI_BASE_URL`; `ANTHROPIC_API_KEY` and `ANTHROPIC_BASE_URL`; `GEMINI_API_KEY` and `GEMINI_BASE_URL`; `PII_MODEL` for a default model; `PII_DISCORD_TOKEN` and `PII_DISCORD_CHANNEL_ID` for the one forum channel the installer configures. Egress is granted per host: the daemon grants `http:<host>` for the host of the configured base URL only, and any other host is denied and reported.

2.7 Supervising the daemon

The daemon is a foreground process: `-foreground` is accepted for supervisors and changes nothing, because the daemon never forks. The installer writes this unit and does not enable it:

```
[Unit]
Description=Pii daemon
After=network-online.target

[Service]
Type=simple
$USERLINE
Environment=PII_RUNTIME_DIR=$RUNTIME_DIR
Environment=PII_CONFIG=$CONF
EnvironmentFile=-$CONFIG_DIR/pii.env
WorkingDirectory=$STATE_DIR
ExecStartPre=/usr/bin/env mkdir -p $RUNTIME_DIR
ExecStartPre=/usr/bin/env ln -sfn $STATE_DIR/graphs $RUNTIME_DIR/graphs
ExecStart=$PREFIX/pii daemon --foreground --config $CONF --store
    ↪ $STATE_DIR/store.redb \
  --workspace $STATE_DIR --log-file $STATE_DIR/pii.log \
  --log-rotate-bytes 10485760 --log-keep-files 5
Restart=always
RestartSec=2
```

```
TimeoutStopSec=10
NoNewPrivileges=yes

[Install]
WantedBy=$WANTED
```

The variables are the installer's own: `$USERLINE` is a `User=` line and is emitted only in the system scope, and `$WANTED` is `default.target` for a user unit and `multi-user.target` for a system one.

Two lines in that unit are load-bearing beyond the obvious. The symlink of the graphs directory runs on every start because the runtime directory is a tmpfs: the graphs live in the state directory and are placed where the daemon looks for them at each boot. `TimeoutStopSec=10` matches the daemon's own shutdown: it waits at most five seconds for in-flight blocking work, then exits anyway, so a stop that needs more than ten seconds is a stuck process and systemd will kill it.

Then enable it, and make it survive logout if the host is meant to run unattended: `systemctl -user daemon-reload`, then `systemctl -user enable -now pii`, then `loginctl enable-linger` for the account. Nothing in the installer runs those three commands, which is deliberate — an install that starts a service is an install you cannot inspect before it acts.

The repository also ships a static unit for the system scope, `deploy/pii.service`, which uses `DynamicUser=yes` with `StateDirectory`, `RuntimeDirectory` and `LogsDirectory` so that `/var/lib/pii`, `/run/pii` and `/var/log/pii` are created, owned and kept across restarts. Use one or the other, not both: two units for the same daemon is the same defect as two daemons for the same store.

2.8 The graphs an installation serves

Graphs are installed as files under `$STATE_DIR/graphs`, and the daemon resolves a name through three directories in order: `<runtime dir>/graphs`, then the `graphs` directory beside the store, then `$HOME/.local/state/pii/graphs`. A directory listed twice is searched once. That order is why an installed copy can shadow a repository copy, and why `pii graph list` prints a drift column: `same`, `DRIFT`, `installed-only` or `repo-only` compares each installed file against its repository twin by the hash of its bytes.

```
pii graph list                  # name, hash, node count, trigger, armed
    ↪ runs, drift
pii graph list --all            # include graphs that are only in the
    ↪ repository
pii graph edit queue --file graphs/queue.toml    # lint, move the old one
    ↪ aside, write atomically
```

`pii graph edit` never deletes a graph: it lints the candidate first, moves the previous file aside, and writes atomically. A graph that fails lint is not installed at all, so a bad edit cannot take out a working graph.

2.9 Verifying the installation

Four commands, in increasing depth. Each has an expected reading, and the exit codes are part of the answer.

```
pii --version                  # the binary you are actually running
pii health                     # 0 = this build, 2 = no daemon, 3 = drift
    ↪ or unverified
pii health --json              # the full sys.status payload, for a script
pii graph list                 # does the daemon see the graphs on disk?
```

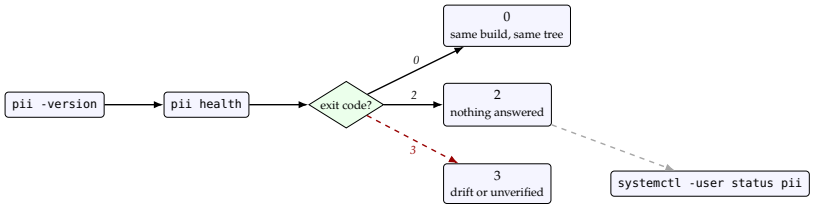


Figure 2.4: The verification ladder, and the three readings of `pii health`. Exit 3 immediately after an install is correct: the client is the new binary and the daemon is still the old process. It becomes 0 only after the restart.

A health answer whose `stores` list is empty is not a healthy daemon, whatever its exit code: it is a daemon pointed at an empty directory. Check `PII_STATE_DIR` and the unit's `-store` argument before believing anything else. The same reading applies to `format_version`: a store at a version the binary does not know is refused at open, with the path and both versions in the message, and nothing is written to the file.

2.10 Rolling an installation back

`deploy/rollback-pii.sh` undoes the last install without deleting anything. It stops the unit if it is running, finds the newest `$PREFIX.old-<epoch>` directory, and moves its contents back — binaries to the prefix, `pii.toml` to the config directory, the unit to the unit directory, and any `graph.*` file to the state directory's `graphs`. Whatever it replaces is moved to `$PREFIX.rolledback-<epoch>`, and it prints each file restored.

```
bash deploy/rollback-pii.sh
systemctl --user daemon-reload && systemctl --user restart pii
pii health                                  # back to the older revision
    ↪ , exit 0
```

The script is exercised in a scratch prefix by the deployment checks, and the drill is worth repeating once on a real host before you need it: an install followed by a rollback followed by `pii health` returning 0 proves both directions, and costs a minute.

2.11 Failure modes of the install itself

Symptom	Cause, and what to check
`ABORT: no deploy card for revision ...`	No `_tmp/deploy/card-<rev>.json` for `HEAD`. Render and approve a card, or install a revision that has one.
`ABORT: llama.cpp host ... does not answer`	The model host is down or the URL is wrong. Nothing was changed; the probe runs before the build.
`cargo: command not found` inside the export	A non-interactive shell did not carry your `PATH`. The script now exports it itself; an older copy of the script is what fails here.
The installed binary reports `<rev>-dirty`	It was built in the working tree. Only a clean-export build is deployable evidence.
The installed binary reports `unknown`	The export build did not receive `PII_BUILD_REVISION`. Rebuild with the script, not by hand.
`pii health` exits 2 after install	The unit was written but not enabled, or it failed to start. `systemctl -user status pii`, then the log file.
The daemon starts but every graph fails at spawn	The model id is one the provider does not serve, or the key is not in the unit's environment.

The last row is the one that looks like a pii bug and usually is not. A model id that the provider has retired fails at spawn, in a second, with no partial output; the spec graph takes its `err` edge to a `refuse` node, and the queue records the row as refused rather than failed. Read the id in the graph, then the model the daemon resolved, then the provider's own list.

Daily use 3

A normal day with pii is short. You ask the daemon what it is, you give it work — a prompt, a graph, a queue row — and then you read what it did. This chapter is that loop in the order you actually run it, with the exact flags the binary accepts at the revision this manual describes, the exit code each command returns, and the places where the answer looks like a success and is not. Nothing here needs the repository; everything here assumes a daemon that is already running, as installed in Chapter 2.

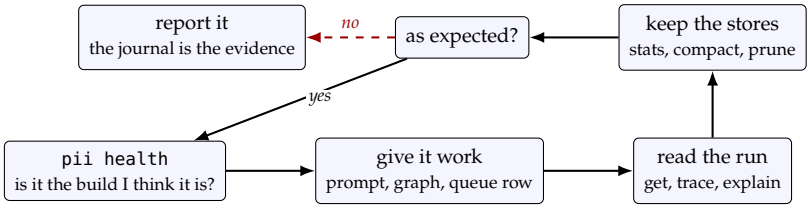


Figure 3.1: The daily loop. The first and last steps are the ones that get skipped: a health call before you trust a result, and a store reading before the disk tells you.

3.1 Asking the daemon a question

`pii health` is the first command of any session, including a session that starts with "it worked yesterday". It prints the daemon's own account of itself and returns an exit code that a script can branch on.

```
pii health                # human text; 0 = same build, 2 = no daemon, 3 = drift
pii health --json         # the sys.status payload, verbatim
pii metrics               # one name-value line per metric
pii disk                  # artefact roots with size, age and filesystem
```

The text form is documented line by line in Chapter 1 and is not repeated here. The useful habit is narrower: read `client:` against the first line, because that pair is the only thing in pii that tells you whether the binary in your shell and the process answering you are the same program. A `DRIFT` line on stderr with exit 3 is not an error to fix; it is a fact to act on — restart the service, or use the binary the daemon is running.

`pii disk` answers a different question, and it exists because the answer used to take four commands and a guess. It walks a fixed list of artefact roots (`_tmp/gate-target`, `_tmp/gate-target-fresh`, `_tmp/targets`, `_tmp/_trash`, `/projects/_tmp`, `/projects/_trash`, `/tmp/_trash-podman`, `/tmp/gate-cache` and every unclaimed `_tmp/*target*` directory), prints each one's size, age, filesystem and whether an owner claims it, then prints free space for `/` and for the volume that holds the repository. It is read-only; removal is `cargo xtask disk -gc`, which owns the TTL.

The reason it prints *two* free-space lines is a measured defect: the gate's floor was evaluated on `/` while the artefacts landed on `/projects`, so a host with 16 GB free on a 288 GB overlay and 147 GB free on the repository volume refused to run — and the obvious first move, `cargo clean` on the repository, freed space on the volume that was never short.

3.2 One-shot prompts

`pii -p` is the shortest path to a model. It carries a prompt, and its flags decide which model, which graph, which workspace and how much confinement.

```
pii -p "summarise the open items in specs/350-an-incident-becomes-a-case.
    ↪ md"
pii -p "why did run r000000000130 end unverified" --model llamacpp-server:
    ↪ granite-4.2-3b
pii -p "check the tree" --graph preflight --workspace /projects/pi-
    ↪ improved
```

Flag	Effect
`-model M`	the model spec for this call; wins over `$PII_MODEL` and over `[core].model`
`-graph G`	run an installed graph instead of the default one-shot path
`-workspace D`	the directory the sandbox and the tools are scoped to; defaults to the current directory
`-sandbox off|workspace`	`workspace` is the default; `off` runs unconfined and says so
`-connect auto|loopback|socket=<path>`	which transport to use; `auto` takes a live daemon socket if one answers
`-json`	machine-readable output; already the default when stdout is not a terminal
`-input J`	extra graph input, merged into the run input as JSON

Two behaviours are easy to observe and worth knowing before the first surprise. The transport is chosen for you: a live daemon socket wins, and only if nothing answers does the command build the engine in its own process. The daemon is never started for you — there is no auto-start anywhere in the path — so a `-p` on a host whose service is down silently becomes an in-process engine with its own view of the state directory. And if no model resolves at all, the command exits 5 with one line: “no model configured (set PII_MODEL or –model)”. That is a different failure from a model the provider does not serve, which fails later, at spawn.

3.3 Running a graph

There are two ways to run a graph, and they are not interchangeable. By *name*, you ask the daemon, which serves the installed set and owns the stores. By *file*, you run it yourself against a store you name, which is what the test suites and a one-off diagnostic do.

```
# by name: the daemon resolves it, runs it, and journals it
pii run start queue --input '{"workdir":"/projects/pi-improved","base
    ↪ ":"<40-hex>"}'

# by file: you own the store path and the write grants
pii graph run graphs/preflight.toml --db ~/.local/state/pii/store.redb \
  --input '{}' --allow-write /projects/pi-improved/_tmp
```

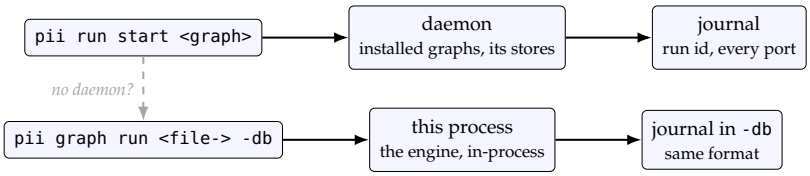


Figure 3.2: Two paths, one journal format. The name path is what an installation uses; the file path is what a diagnostic uses, and it is the only one where you choose the store.

The offline path builds its own capability root, and the defaults are worth memorising because they explain most "permission denied" results from a hand-run graph: reads are allowed for the current directory, writes are allowed for `<cwd>/_tmp` plus every `-allow-write` directory, and the only process argv[0] permitted is `/bin/sh`. Everything else is denied by the capability check before the node runs. `-allow-write` is canonicalised, so a relative or symlinked grant would otherwise grant nothing.

Both paths share one exit-code vocabulary, and the two codes that are not 0 or 1 are the interesting ones:

Code	Meaning
0	the run reached its terminal node and reported success
1	the run failed, or could not be started; the message names the reason
2	usage error: an unknown flag, a missing value, `-input` that is not JSON
3	the validator refused the graph — and, on this path, no store file was created
4	the run suspended and holds a resume key

The distinction between 3 and 1 is the reason a refused graph creates no store file: the graph is loaded and validated *before* the store is opened, so a wrong pin or a missing callee costs nothing and leaves nothing behind.

3.4 The queue

The queue is how pii is given work rather than a question. A row carries an id, a title, a spec path, the criteria that decide whether the work is done, and the paths the row expects to touch. Every subcommand is one protocol call; none opens the store.

```
pii queue add --id 376 --title "retention drops what the policy expires" \
  --spec /projects/pi-improved/specs/376-retention-drops-what-the-policy-
    ↪ expires.md \
  --criteria "cargo test -q -p pii-store --test
    ↪ retention_drops_what_the_policy_expires" \
  --touches crates/pii-store/src/retention.rs
pii queue ls
pii queue claim --by operator
```

The spec path is resolved by the daemon, not by your shell. A relative path is the single most common first mistake with `queue add`: the daemon's working directory is the unit's `WorkingDirectory`, which is the state directory, so a relative `specs/376-*.md` resolves to a file that does not exist. Use an absolute path and the row will be claimable.

Running the queue is one graph run, and the graph needs two inputs: the work directory and the commit that holds the row's spec. Both are passed as one JSON object to `pii run start queue -input`, with `workdir` an absolute directory and `base` a forty-hex commit. The spec graph's own limits are shared by the call, so `max_wall_ms` on the queue's `[limits]` table is the whole run's wall, not each step's.

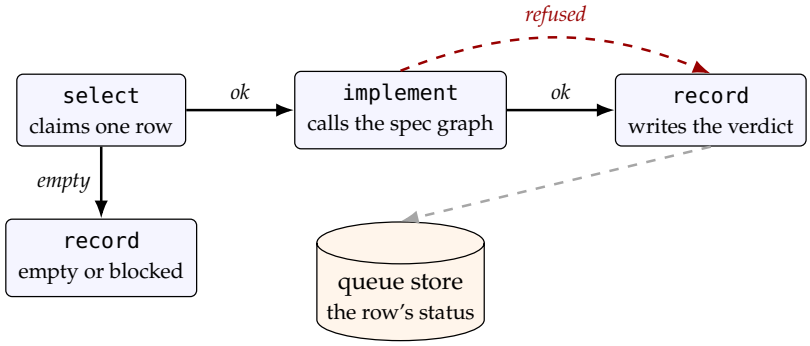


Figure 3.3: `queue` claims one row, hands it to the implementer, and records a verdict. The red edge is the one that took a row from "stuck Running" to "recorded as refused".

The three ports of `select` are the whole reason the queue is readable. `ok` means a row was claimed, `empty` means there was nothing to do, and `blocked` means a row exists but something it waits on does not — and `blocked` ends the run on its own edge rather than pretending to have worked. A row that stays `Running` after the run ends is the failure this shape exists to prevent: at the revision this manual describes, a refused callee used to dead-end inside the splice because the callee's refusal nodes declare no `ok` port, so no edge fired and the row was never recorded. The fix wires the refusal by name — `implement.refused`, `implement.unverified` — and the queue's own graph now carries both edges.

The rule for an operator is one sentence: if a row does not leave `Running`, read the trace and look at the last node, not at the last message. The verdict is in the journal even when the record was never written.

3.5 Reading a run

Three commands read a run, and they answer different questions.

```
pii run ls --status suspended          # running|suspended|succeeded|failed
pii run ls --graph queue               # filter by graph name, client-side
pii run get <run_id>                   # the header: graph, status, timing
pii run trace <run_id>                 # node by node, with the port that
    ↪ fired
pii run explain <run_id>               # the same journal, summarised
```

`-status` accepts exactly four names and refuses anything else with the list in the message. `refused` is a run status the store knows but not one the filter accepts, which is a small asymmetry worth knowing before you script it.

`pii run get` distinguishes three answers that look alike in a hurry. A run the daemon holds but has not journalled yet answers `accepted` — the id exists and the journal has not materialised, so you poll again. A

stored state answers `running`, `suspended`, `succeeded` or `failed`. An id the daemon has never heard of answers "unknown to this daemon": it was never issued, or the daemon restarted and lost an accepted-but-never-journalled start. Polling an `accepted` id until it becomes terminal is correct; treating the first `not found` as a failure is not, and the measured baseline is that a validated start never polls `not found` for ever — a start the run's own validation refuses still returns an id, then goes to `failed` with the reason in the trace.

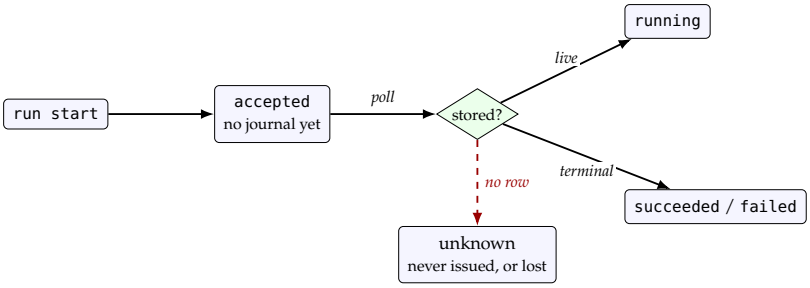


Figure 3.4: Three answers from `run get`, and only one of them is a polling state. The red edge is the reading that is a real problem: an id the daemon never issued, or lost across a restart before its journal was written.

Read a trace from the end. The last `NodeFinished` names the node and the port it took, and that pair is the decision the run made. A run that stopped without reaching a terminal node has not failed loudly — it has failed quietly, which is the failure mode the journal exists to make visible. The events you will read most are `Started`, `TickStarted`, `NodeStarted`, `NodeFinished`, `NodeFailed`, `NodeProgress`, `NodeCancelled`, `NodeDead`, `NodeInterrupted`, `Suspended`, `Resumed` and `NodeObserved`; Chapter 1 lists all seventeen.

`NodeProgress` deserves its own sentence, because it is the only event that separates a slow node from a wedged one: it carries elapsed milliseconds and the child's CPU milliseconds, so *elapsed grows, CPU flat* is a node waiting on something that will never answer, while *both grow* is a node doing work.

The offline trace, and its queries

When the daemon is down, the same journal is readable from the store file directly, and the offline form answers more questions than the online one.

```
pii graph trace <run_id> --db ~/.local/state/pii/store.redb
pii graph trace <run_id> --db ... --terminal
pii graph trace <run_id> --db ... --node-status verify
pii graph trace <run_id> --db ... --reason refused
pii graph trace <run_id> --db ... --overlapped code,docs
```

With no query at all, the command prints the journal as NDJSON: one event per line, each with a sequence number. `-terminal` prints the run's terminal value, `-node-status` prints one node's last port, `-reason` prints why a refusal node refused, `-max-batch-width` reports how wide the scheduler's batches ran, and `-overlapped a,b` answers whether two nodes were ever in flight at the same time — which is the question you ask when a graph is suspected of hiding a serialisation.

When a run needs a hand

Four commands exist for runs that will not finish on their own, and they differ in what they do to the evidence.

Command	What it does
`pii run resume <id>`	continues a `Suspended` run through its resume key
`pii run retry <id-> -node <id>`	re-runs one node of a finished run
`pii run abandon <id-> -reason T`	marks the run abandoned, with the reason recorded
`pii run cancel <id-> [-kill]`	cancels a live run; `-kill` also kills the child

Abandon and cancel are the two that change what retention will later do. A run left `Running` or `Suspended` is never swept, however old it is, so a run you have given up on stays in the store for ever unless you end it deliberately.

3.6 Stores, memory and the indices

Store maintenance is three commands, and two of them need the daemon stopped. `pii store stats` prints one line per table with its row count, then the file's byte size. `pii store compact` takes redb's exclusive lock, so a running daemon makes it refuse rather than copy a file mid-write, and it prints the measured size of each file before and after. `pii store prune` takes the same lock, runs one retention sweep and prints `pruned N rows`.

`pii store stats` is the command to run weekly. It prints `store.redb <table-> rows=N` for every table and then `store.redb bytes=N`, which together are the only honest growth signal — the file size alone is not, because redb never shrinks and a store that has been compacted tells you nothing about how fast it is filling. `pii store compact` prints `compacted <name-> <before-> -> <after-> bytes` per file, and a store with nothing to reclaim prints a near-equal pair, which is the truth rather than a failure.

`pii store prune` runs one retention sweep under the default policy and prints the count. The defaults are thirty days for finished runs, seven for node-call events and at most one thousand rows per sweep; the sweep never removes a `Running` or `Suspended` run, and it keeps any row whose creation time is missing. The daemon also runs one sweep at boot, so a host that restarts nightly trims itself.

Memory has its own command group, and the two that are used daily are search and the entity graph: `pii memory search <q>` with an optional `-k` (six results by default), and `pii memory put <text>` with `-layer` and `-kind` for a document's tier. `pii memory tiers` says what sits in each tier, `pii memory entity list` gives the entities and their counts, and `pii memory neighbours` gives one hop around an entity. The rest of the group (`learn`, `consolidate`, `link`, `compact`, `archive`, `restore`) is maintenance, and each one either refuses or reports when its precondition is missing rather than doing half the work.

`pii index code` and `pii index docs` fill the code and documentation indices from a tree, and `pii skill list` answers what the daemon knows how to do. Both are read-mostly and safe to re-run: an index command re-reads and updates, it does not duplicate.

3.7 A cadence that works

The commands above are cheap; the discipline is knowing which ones to run on a clock. The cadence below is the one Chapter 4 assumes, written as a table so it can be checked rather than remembered.

When	Command	What you are looking for
Every session	`pii health`	exit 0; `client:` equals the build line
Every session	`pii run ls -status running`	a run that should have finished
Daily	the log file, last lines	a log that ends on `daemon.started` means a hard death
Weekly	`pii store stats`	rows and bytes per table, recorded, so the slope exists
Weekly	`pii disk`	an artefact root growing without an owner
Monthly	`pii store compact`	the tail slack back; nothing more
Monthly	a restore into a scratch directory	a backup nobody restored is a hope

Two of those rows are worth the cost even on a quiet host. A weekly `store stats` sample is what turns "the disk is filling" into a rate, and a monthly restore into a scratch state directory is what turns a backup into a backup — both drills are described in Chapter 4.

3.8 What a quiet day looks like

A host with nothing to do is not silent, and knowing the small noises saves a false alarm. The daemon writes a `daemon.started` event, keeps its health record current, and runs one retention sweep at boot. If a channel is configured it polls on its interval; if a worker is configured it says hello once. Nothing else should appear in the log.

The four readings that answer "is this host doing anything" are `pii health` (first three lines: build, client, config hash), `pii run ls` (usually an empty list on a host with no work), `pii cron ls` (the schedules the daemon knows) and the last three lines of `$STATE_DIR/pii.log`. On a healthy host the log's last line is a `daemon.started` only after a restart — a log that ends on `daemon.started` while the process has been up for days means the daemon died hard and was restarted by the unit, and `since_deploy.hard_deaths` in `pii health` will say how many times.

If `pii run ls` is empty and you expected rows, the question is not "where did the runs go" but "which store is this daemon writing" — and `pii health` answers it with a path. That single habit, reading the store path before believing an empty list, resolves more "lost work" reports than any other diagnostic in this manual.

4 Operating

This is the chapter nobody reads until something has gone wrong, and it is written for that moment: short procedures, the exact command for each, the reading that tells you whether it worked, and the failure modes that have actually happened on this product rather than the ones a design review imagined. The order is the order of the questions an operator asks — can I get the data back, what is the store doing to the disk, what changed when I upgraded, where is the evidence, and why is the daemon not answering.

4.1 Backup and restore

A backup is offline by construction. `pii backup` opens each store with redb's exclusive lock and copies the file while holding that lock, so a running daemon makes the backup *refuse* — with the path and the words "stop the daemon first" — instead of copying a file with a writer mid-commit. Each copy is then re-opened as a redb database and hashed, so a torn or truncated copy is an error rather than a silent bad backup.

```
systemctl --user stop pii
pii backup /backups/pii-$(date +%F) --dir ~/.local/state/pii
systemctl --user start pii
```

The command prints one line per file, with these field names and no others: `backed up store.redb (<bytes-> bytes, format_version=<n->, blake3=<hash>)`. The same facts go into `MANIFEST.json` inside the destination, and the manifest is what a restore checks against. The destination must be new or empty.

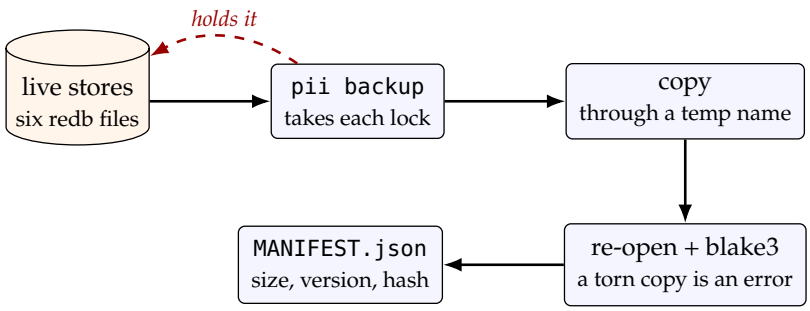


Figure 4.1: A backup that cannot tear. The red edge is the normal case on a live host: the open fails, the command refuses, and no file is written.

Two details decide whether the backup is worth having.

First, the backup set is six files: `store.redb`, `memory.redb`, `approval.redb`, `heal.redb`, `channel.redb` and `timer.redb`. `pii health` reports seven stores, and the seventh — `queue.redb`, where the work rows live — is not in the backup set at this revision. If you rely on the queue, copy that file with the daemon stopped as well, or record the rows somewhere you can rebuild them from.

Second, `-dir` names the directory that holds the stores, and the durable pair (channel, timer) defaults to `<state dir>/durable`, not to the directory beside the main store. A backup run with `-dir <state dir>` therefore captures the main store, memory, approvals and heal, and reports nothing for channel and timer because those files are one level down. Run a second backup with `-dir <state dir>/durable` if cursors and timers matter to you — and they matter the first time a channel replays from the start after a restore.

Restore verifies every file's hash against the manifest before touching the target, never deletes, and refuses a store that is in use.

```
systemctl --user stop pii
pii restore /backups/pii-2026-11-04 --dir ~/.local/state/pii           #
    ↪ refuses if a target exists
pii restore /backups/pii-2026-11-04 --dir ~/.local/state/pii --force   #
    ↪ moves the old store aside
systemctl --user start pii
```

`-force` does not delete: it moves the existing store to `<name>.redb.old-<ms>` and leaves the decision about the old file to you. That is deliberate — the one file you are most likely to want after a bad restore is the store the restore replaced.

4.2 Retention

Retention is the only code in the product that deletes run data, so its rules are worth knowing before you need them. The policy has three numbers and three guards.

Setting	Default and meaning
run age	30 days: a finished run's header, journal and snapshots go past this age
node-event age	7 days: a finished run's node-call events go past this age
rows per sweep	1 000: the sweep's whole budget, so a first run on a large store cannot hold a write transaction for minutes

The guards are the part that matters:

1. A run whose status is `Running` or `Suspended` is never removed, however old it is, because the daemon may still resume it.
2. A row that cannot be dated is kept. A run created before the clock was recorded carries `created_ms` of zero, and deleting on a missing timestamp would destroy exactly the runs an operator is looking for.
3. The sweep is bounded per call, so it is a trim rather than a migration.

One sweep runs when the daemon starts, and `pii store prune` runs one on demand. The command prints `pruned N rows` and exits 0; the store also records a `store.retention.swept` event carrying the count, but only when something was removed. A sweep that removes nothing is silent, which is why the count on stdout is the number to read.

4.3 What retention actually removes

The three rules have an order of precedence, and the order is not the one people assume. Age selects candidates; a live run is excluded; an undatable row is excluded; then the per-sweep budget truncates whatever remains.

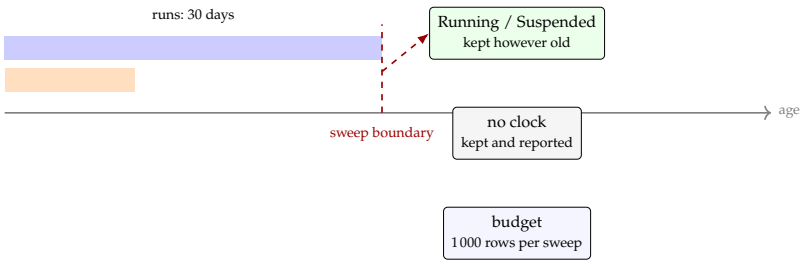


Figure 4.2: Three rules, in order of precedence: age bounds the sweep, a live run is never swept, and a row that cannot be dated is kept. The budget is last, so a large store trims over several sweeps rather than one long transaction.

A run that will never finish is therefore never collected. If you see one, park or cancel it deliberately — `pii run abandon` with a reason, or `pii run cancel` — and retention will collect it thirty days after it ended. Retention will not make that decision for you, and should not: the difference between trimming and losing work in flight is this guard.

4.4 Compaction is not retention

These two get confused, and the confusion costs a maintenance window that reclaims nothing. Compaction reclaims free pages *inside* a file; retention removes rows. redb never returns space to the filesystem except at the end of the file, and only compaction moves that boundary.

The measured pairs from this product's own runbook are the honest expectation:

Store state	Before → after
fragmented: 20 sessions of 400 KB, every other one deleted	13 111 296 → 6 623 232 bytes
a store after 400 runs through a daemon	1 048 576 → 786 432 bytes
idle auxiliary stores (memory, approval, heal, channel, timer)	1 056 768 → 36 864 bytes each

So on a store that has only grown with runs, expect the tail slack back and nothing more. The 400-run row is the one to remember: it is a two-hundred-kilobyte improvement on a megabyte file, which is not a reason to schedule an outage. The auxiliary row is the reason to run compaction at all — five stores that were preallocated and never wrote shrank by about 97 %.

4.5 The restore drill

A backup nobody has restored is a hope. The drill is short, it uses a scratch state directory, and it is the only way to know that the manifest, the hashes and the binary's format version all agree.

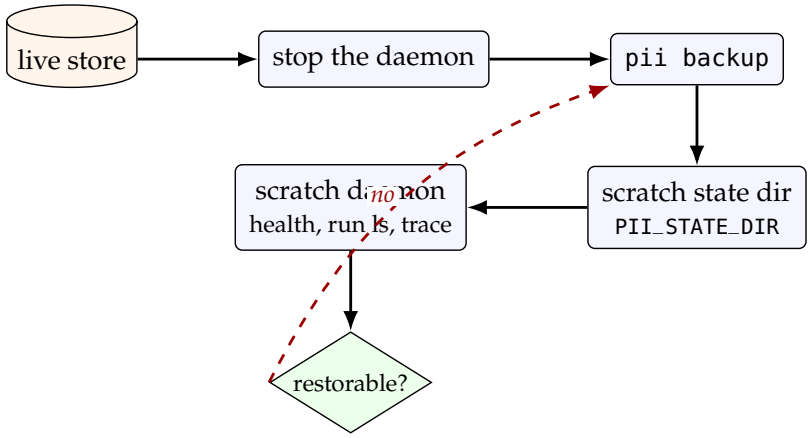


Figure 4.3: A backup is a backup once it has been restored into a scratch state directory and read there. Restoring over the live store destroys the evidence you would need if the restore is wrong.

```
mkdir -p /projects/_tmp/restore-drill
pii restore /backups/pii-2026-11-04 --dir /projects/_tmp/restore-drill
PII_STATE_DIR=/projects/_tmp/restore-drill pii store stats
PII_STATE_DIR=/projects/_tmp/restore-drill pii graph trace <run_id> \
  --db /projects/_tmp/restore-drill/store.redb --terminal
```

Three readings close the drill. `store stats` prints a row count per table, so an empty journal is visible immediately. A trace on a known run id prints that run's terminal value, which proves the journal survived the round trip. And a daemon started on the scratch directory with a different runtime dir proves the binary can open the restored format version — the one thing a restore cannot repair is a store written by a newer build.

4.6 Upgrading

An upgrade is four steps and one expected non-zero exit code. The exit code is the part that alarms people: immediately after installing a new binary and before restarting the service, `pii health` exits 3 and prints a `DRIFT` line, because the client is the new build and the daemon is still the old process. That is the instrument working. It returns 0 only after the restart.

```
systemctl --user stop pii
pii backup /backups/pii-pre-upgrade --dir ~/.local/state/pii
bash deploy/install-pii.sh                                  # card-
    ↪ gated, moves the old aside
pii health                                                  # expect 3
    ↪  + DRIFT
systemctl --user restart pii
pii health && pii --version                                 # expect
    ↪ 0, same revision
```

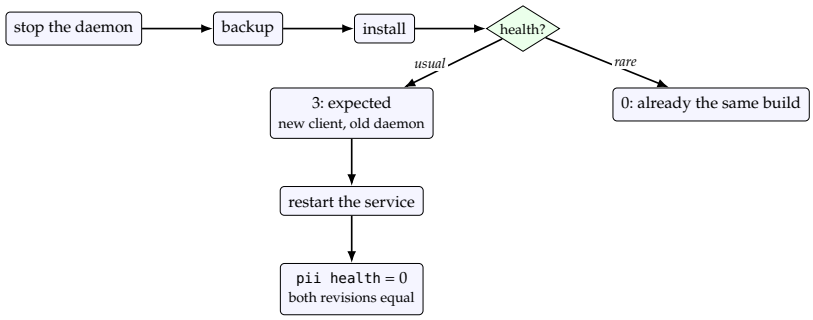


Figure 4.4: The upgrade sequence, with the reading that is correct and looks wrong. A `3` here is not a failed upgrade; it is the proof that the install replaced the binary and the service has not yet been restarted.

The state directory is forward-compatible only in one direction. Every store carries a format version stamped on create or first open, and a binary that meets a newer version refuses to open it:

```
error: invalid: store '/var/lib/pii/memory.redb' has format version 99,
    ↪ which is newer
than the version 1 this binary understands; upgrade pii or use a different
    ↪  store path
```

The daemon exits 2 and writes nothing to the file. The remedy is a binary that understands that version, or a restore of a backup taken by the older build — never an edit of the stamp.

Rollback is the install script's inverse and it moves rather than copies: `bash deploy/rollback-pii.sh` stops the unit, restores the newest `$PREFIX.old-<epoch>` contents, keeps what it replaced under `$PREFIX.rolledback-<epoch>`, and prints each file. Then `daemon-reload` and `restart`, then `pii health` for a 0.

The gate the card quotes

No install runs without a deploy card for the revision being installed. The card is a JSON file with five required fields — `incident` (id, fingerprint, count), `change` (branch, revision, diff stat), `proof` (red on base, green on branch), `gate` (verdict) and `rollback` (rehearsal) — and a card missing any of them is refused with the field named and no approval path printed. Approval is bound to the revision the card names, and every decision appends one line to the operator record (`PII_ACTS`, default `_tmp/autonomy/operator-acts.log`).

The gate verdict has a condition attached that is easy to miss: a deploy run must quote a *fresh-tree* gate, and a `mode=routine` verdict is not release evidence. A routine gate runs in the working tree against a warm target; a fresh gate exports `HEAD` and builds in a throwaway directory. When the two disagree, believe the fresh one — a gate can be green in a tree and red in a clean clone.

4.7 Logs, audit, and the death that leaves no line

The daemon writes events as JSON lines, one per event, each stamped with `ts_ms`. The log is off unless a path is set. Three keys control it, and

each has a flag that overrides it.

Key	Default	Flag
`[core].log_file`	absent: no log file	`-log-file`
`[core].log_rotate_bytes`	10 485 760 (10 MiB); 0 never rotates	`-log-rotate-bytes`
`[core].log_keep_files`	5 rotated files besides the live one	`-log-keep-files`

Rotation renames rather than truncates: `pii.log` becomes `pii.log.1`, `pii.log.1` becomes `pii.log.2`, and the oldest is discarded, so disk use is bounded by about `(keep + 1) * rotate_bytes`. The file is rotated *before* a line would push it past the limit, so a single pathological line can exceed the bound once.

Denials are separate and always on: they go to `audit.log` in the store directory, rotated the same way, whatever the log file is set to. If you are investigating a permission problem and the daemon log is clean, read `audit.log`.

The one reading that matters is the last line. A log that ends on `daemon.started` with no `daemon.stopped` means the process died hard — a `kill -9`, an OOM kill, a machine reset. `pii health` agrees: `since_deploy.starts` and `since_deploy.hard_deaths` are counters kept across restarts, so a host that has restarted eleven times and died hard twice says so in two lines.

4.8 Disk is owned, bounded and visible

Disk needed its own command because the obvious answers are about the wrong filesystem. The measured incident: the container's `/` is a 288 GB overlay that reached 16 GB free (95 %), while `/projects` — a separate 594 GB volume — had 147 GB free. The gate refuses to run below a 12 GB floor on `/`, so work stopped, and `cargo clean` on the repository freed 152 GB on the volume that was never short.

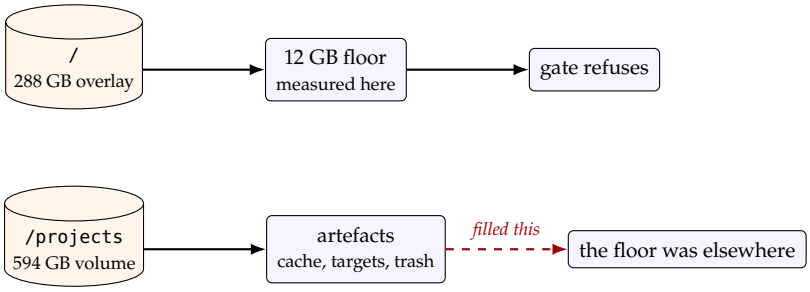


Figure 4.5: The mismatch that hid the growth: the floor was measured on `/`, the artefacts landed on `/projects`. Freeing the volume looked like progress and was worth nothing.

`pii disk` prints the artefact table and *both* free-space lines, which is the whole fix for that class of confusion. Four mechanism families fill the disk, and each has an owner now:

Mechanism	Owner and bound
the warm gate cache	ceiling of 40 GiB (`FRESH_GATE_CACHE_MAX_GB`); trimmed before a build, never while a build is live. It once reached 105.8 GiB because nothing bounded it.
one-off experiment targets	`_tmp/targets/<purpose>` with a TTL of 7 days (`DISK_TTL_DAYS`), removed by `cargo xtask disk -gc`
trash	dated directories with a retention window, not a build artefact
failed gate exports	removed after `DISK_EXPORT_TTL_HOURS` unless a gate still owns one

`cargo xtask disk -gc` refuses to remove anything while a cargo or rustc is live and prints why, which is the guard that keeps the cleanup from becoming the incident. The 12 GB floor is not lowered to make a gate pass; if the floor is what stops you, the answer is space.

4.9 When the daemon is wedged

The sequence is short and it is ordered so that no step destroys evidence the next one needs.

1. `pii health -json` with a short timeout. No answer within a few seconds means the daemon is wedged or dead, and exit 2 means nothing is listening at all.
2. Read the last lines of the log file before stopping anything. A `daemon.started` with no `daemon.stopped` is a hard death; the last non-start event names what it was doing.
3. `systemctl -user stop pii`. Shutdown waits at most five seconds for in-flight blocking work, then abandons it and exits anyway. If something was stuck, stderr carries exactly one line: "pii: shutdown drain exceeded 5s; abandoning in-flight blocking work". That line is stderr only — it is in the journal, not in the log file.
4. If systemd had to SIGKILL it, nothing is lost structurally: the store is a transactional redb file, and the daemon reconciles interrupted runs on boot. Start it and read `pii health`.
5. A run that was mid-flight comes back as `Running` or `Failed`, with a `NodeInterrupted` event in its journal for the node that was left in flight. Read the trace before resuming it.

4.10 Failure modes that have actually happened

Symptom	Cause and what to check
Every graph run fails in a second, with no output	A model id the provider does not serve. The old id had not existed for months and the graph died at spawn; run `pii graph lint` and read the model in the graph.
The daemon answers, but the store is empty	Two different `PII_STATE_DIR` values — one for the daemon, another for your shell — or a daemon whose `-store` points at the runtime directory. Read the path in `pii health`.

Symptom	Cause and what to check
A row stays `Running` for ever	A callee refused and no edge carried the refusal. At this revision the queue wires `refused` and `unverified` by name; an older graph dead-ends inside the splice. Trace the run; the refusal is in the journal.
The coder stops and says no write tool is available	The graph's `caps` omit `fs_write`, so the node may not hold it. That is the capability ladder working, not a bug.
The gate refuses with a disk message on a full disk	The floor now names the filesystem it measured. A message that names only `/` while the artefacts land elsewhere is an older binary.
`backup` refuses with "cannot be opened exclusively"	The daemon is running and holds the lock. Stop it; the refusal is the no-tear guarantee, not a fault.
`health` prints `UNVERIFIED: daemon build revision is unknown`	The running binary was built without its revision — an export build that did not receive `PII_BUILD_REVISION`. Rebuild with the installer.
A channel replays its backlog after a restore	The channel cursor lives in `durable/channel.redb`, and a backup of the store directory does not include it. Back that directory up too.

4.11 The first five minutes of an incident

Order matters more than completeness. Five checks establish whether you are looking at a process problem, a data problem or an environment problem, and each takes seconds.

Step	Command	What it settles
1	`pii health`	Is anything answering, and is it this build? Exit 2 is a process problem; exit 3 is a deployment problem.
2	`pii run ls -status running`	Is work in flight? A run that should have finished is a graph problem, not a daemon problem.
3	`pii disk`	Is the host out of space on the filesystem the artefacts use? A gate or a write failing at 95 % looks like a code defect.
4	the log's last ten lines	Did it die hard, and what was the last thing it did?
5	`pii store stats`	Is the store growing at a rate that explains the symptom, and does the row count match what you expect?

The order is not arbitrary. Checking the store before the disk gives you a row count you cannot interpret; checking runs before health gives you a list from a daemon that may not be the binary you think. Health first, because every later reading depends on knowing which process produced it.

4.12 Two rules that keep a host boring

The rest of this manual is detail; these two rules prevent most of the incidents in it.

Never point a second process at a live state directory. The stores are exclusive, the daemon claims the durable pair before it binds, and a second daemon will be refused — but a CLI command with a different `PII_STATE_DIR` will not be refused, it will quietly read a different store and report an empty world. If a reading surprises you, the first question is which store it came from.

Never delete what you can move. The installer moves old binaries aside, the restore moves the replaced store aside, retention keeps undatable rows, and compaction refuses while a writer holds the file. Every one of those behaviours exists because the alternative destroyed something nobody could reconstruct. When a procedure in this chapter seems to be missing a cleanup step, that is the step it is missing on purpose.

Command reference 5

This chapter is the complete surface of the `pii` binary: every group the dispatcher accepts, the verbs inside each group, the flags each verb reads, and — the part that is usually missing from a command reference — what the verb does to the stores on disk. A flag is easy to look up; the difference between a command that asks the daemon to write and one that opens the same file behind the daemon's back is what corrupts an installation.

Every group below was read from `crates/pii-cli/src/` and matched against the captured help output. Where a group has no help text of its own (several have none), the flags were read from the parser, which is the authority.

5.1 The shape of the surface

The first argument selects a group; everything after it belongs to the group. An unknown first argument prints the usage string and exits 2. There is no plugin mechanism and no argument-file syntax: what the binary accepts is what this chapter lists.

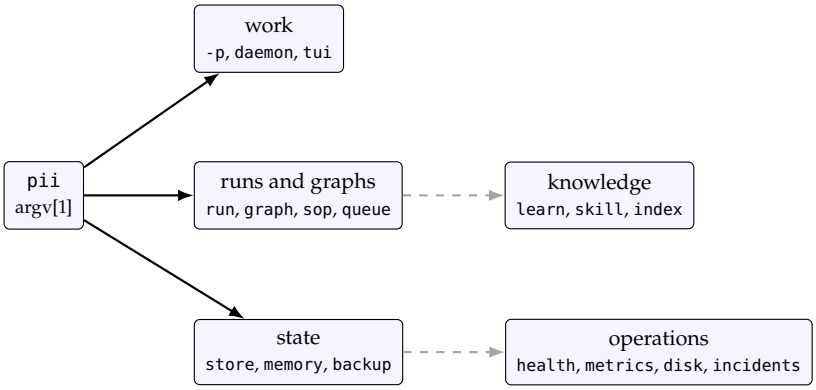


Figure 5.1: Thirty-three groups plus `--version`. The grouping is by what the operator is doing, not by which crate implements the verb.

Three paths to state exist, and a command takes exactly one of them.

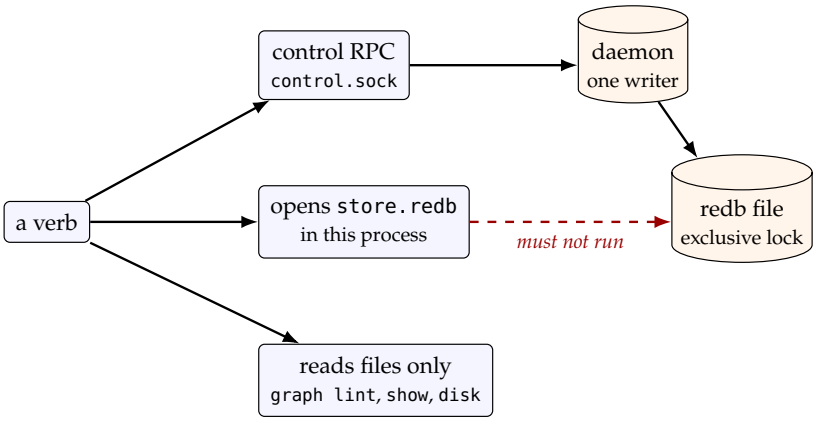


Figure 5.2: The rule behind every "daemon must be stopped" message: redb takes an exclusive lock, so a verb that opens the store directly and a daemon that already holds it cannot both proceed.

Group	Verbs	Path to state
`--version`	none	none
`-p`	one prompt, options	daemon if up, else in-process
`daemon`	foreground server	owns the stores
`graph`	`lint`, `check`, `show`, `run`, `resume`, `trace`, `list`, `explain`, `edit`	files, or `-db` directly
`sop`	`compile`, `lint`, `show`, `run`, `resume`	`-db` directly
`run`	`ls`, `start`, `get`, `trace`, `explain`, `resume`, `retry`, `abandon`, `cancel`	RPC
`queue`	`add`, `ls`, `claim`	RPC
`store`	`compact`, `stats`, `prune`	`store.redb` directly
`backup`, `restore`	one positional each	copies the store files
`memory`	`search`, `put`, `learn`, `consolidate`, `entity`, `link`, `neighbours`, `compact`, `show`, `tiers`, `archive`, `restore`	RPC, or `memory.redb`
`approve`	`ls`, `allow`, `deny`	RPC
`heal`	`ls`, `show`	RPC
`cron`, `net`, `worker`, `sys`	`ls`; `status`, `reload`	RPC
`health`	none, `--json`	RPC
`metrics`	none	RPC
`disk`	none, `--root`	filesystem only
`incidents`	`scan`, `show`, `distance`	files, or a store dir
`loop`	`emit`, `events`	one NDJSON file
`deploy-card`	`render`, `approve`, `refuse`	card file and an acts log
`learn`	`scan`, `list`, `show`, `verify`, `promote`, `promotions`, `revert`, `ledger`	`store.redb`
`skill`	`add`, `list`, `show`, `explain`, `edit`	RPC, or `memory.redb`
`index`	`code`, `docs`	RPC

Group	Verbs	Path to state
`mqtt`	`ls`, `sub`, `pub`, `send`	broker, not the store
`treat`	`export`, `import`, `apply`, `list`	healer store and files
`lane`	`plan`, `run`, `rev`	writes a plan, starts nothing in `plan`
`soak`	none, flags only	RPC against a socket
`tui`	none	loopback or socket
`sandbox-exec`, `worker-exec`	`argv`, frames	process only
`project`	`init`	writes a scaffold

5.2 Starting work

`pii --version`

One line, the version compiled into the binary. It is the only group that touches nothing.

`pii -p "<prompt>"`

The headless entry point. It prefers a live daemon socket; without one it builds the engine in this process and serves the same connection over a loopback duplex. It never starts a daemon. When stdout is not a terminal, JSON is the default output format.

Flag	Effect
`--json`	machine-readable result; the default when stdout is not a TTY
`--model M`	the model spec for this call, overriding `[core].model`
`--graph G`	run graph `G` instead of the agent loop
`--input J`	JSON merged into the run input (graph parameters)
`--workspace D`	the directory tools and the sandbox are scoped to; default cwd
`--sandbox off|workspace`	confinement profile; `workspace` is the default
`--connect <mode>`	transport: `auto`, `loopback`, or a socket path; default `auto`

`pii daemon`

The server, in the foreground. It holds the store locks, binds `control.sock`, loads the graphs, and starts the cron and channel machinery. It never forks, which is why `--foreground` is accepted and does nothing: supervisors pass it out of habit.

Flag	Effect and default
`--store <path>`	redb store file; default `<runtime dir>/store.redb`
`--config <path>`	TOML file; default `$PII_CONFIG`, else `$HOME/.pii/pii.toml`
`--workspace <dir>`	root the sandbox and tools are scoped to; default cwd
`--sandbox <p>`	`off` or `workspace`; default `workspace`, exit 4 if it cannot be enforced
`--log-file <path>`	append events as JSON lines; default off
`--log-rotate-bytes <n>`	rotate past `n` bytes; default 10485760
`--log-keep-files <n>`	rotated files kept beside the live one; default 5
`--observer on|off`	detach the observer fan-out; measurement only, default on
`--audit on|off`	stop the denial feed; measurement only, default on

`--sandbox workspace` confines writes to the workspace, the store directory, the runtime directory and the configured log and trajectory directories; reads are not confined. `--sandbox off` is the explicit opt-out and the daemon says so at start with a warning event. A profile that cannot be enforced is a startup failure with exit 4, not a degraded start: an unconfined daemon that believes it is confined is worse than one that refuses.

`pii run` — through the daemon

Verb	Effect
`ls [--status S] [--graph G]`	list run headers; `S` is one of `running`, `suspended`, `succeeded`, `failed`. The graph filter is applied by the client, the status filter by the daemon.
`start <graph> [--input J]`	begin a run; invalid JSON is a usage error, never a silent empty input
`get <id> [--db P]`	the header, plus one line per agent call. With no daemon it falls back to the store at `--db` or `$PII_RUNTIME_DIR`
`trace <id> [--db P]`	the journal, node by node
`explain <id> [--db P]`	the journal read as a narrative
`resume <id>`	continue a suspended run from its journal
`retry <id> --node N`	re-run one node, keeping the journal
`abandon <id> --reason T`	close the run without continuing it
`cancel <id> [--kill] [--reason T]`	stop a run; `--kill` stops an executing node and needs no reason of its own

`run get` distinguishes two answers that used to read alike: a run the daemon holds but has not journalled yet is “accepted and in flight”, and

an id the daemon has never heard of is “unknown to this daemon”. The distinction matters during a restart: the first is a run to poll, the second is a typo or a wrong store.

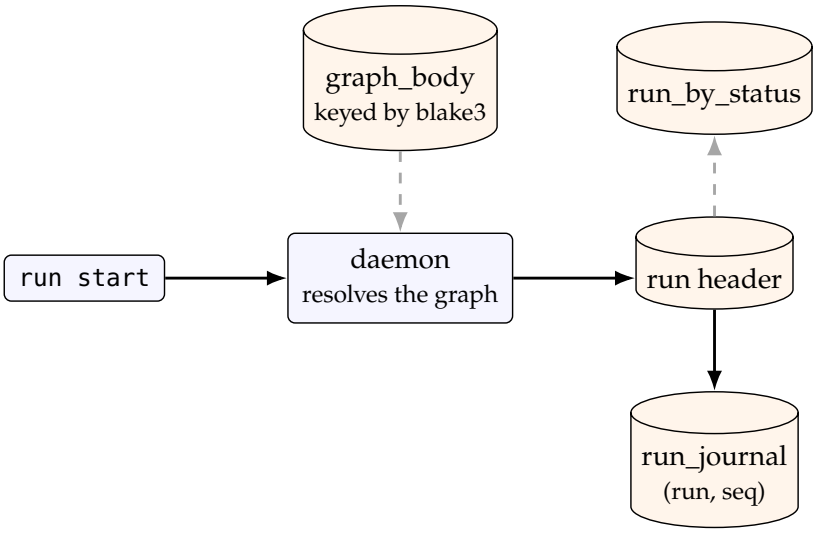


Figure 5.3: What `run start` writes. The graph body is stored once per revision and the header points at its hash, so a run remains readable after the graph file changes.

`pii graph` — without the daemon

`graph run` and `graph resume` execute in this process against a store named by a mandatory `--db`. They are the offline path: no socket, no daemon, one writer. The graph is loaded and validated *before* the store is opened, so a refused graph leaves no store file behind.

Verb	Flags and effect
`lint <file.toml>`	static checks; prints `OK` plus each inlined node's surface, order and origin. Exit 3 on validation errors, 2 on a read or parse failure.
`check <file.toml>`	the spec-207 name for `lint`; same code path
`show <file.toml>`	validate, then print the ASCII topology. A graph that does not validate is never rendered.
`run <file> --db P`	execute and journal. Also `--run-id`, `--input`, `--allow-write <dir>` (repeatable), `--no-validate`
`resume <id> --db P`	replay and continue; `--allow-write` is repeatable
`trace <id> --db P [QUERY]`	eight queries over the stored journal (see below)
`list [--all] [--json] [--dir D] [--repo R]`	installed graphs, their revision, their armed-run count and their drift against the repo copy
`show <name> [--dir D]`	the installed definition, when the name is an installed graph
`explain <name> [--dir D] [--repo R]`	nodes and ports in execution order
`edit <name> --file F [--force]`	lint, move the previous file aside, write atomically. No verb deletes a graph.

The trace queries are worth memorising, because each answers a question that reading the dump by eye answers slowly:

Query	Answers
(none)	the whole journal, one line per event
`--count-finished N`	how many times node `N` finished
`--node-status N`	the last status recorded for `N`
`--max-batch-width`	the widest batch the scheduler executed
`--terminal`	the run's terminal word
`--output`	the run's final output value
`--overlapped A,B`	whether two nodes were ever in flight together
`--reason N`	why node `N` ended where it did

`pii sop`

A declarative SOP is compiled into the same graph engine; `run` and `resume` use the same journal as `graph run`. There is no second runner.

Verb	Effect
`compile <f.sop.toml>`	print the graph the SOP compiles to
`lint <f.sop.toml>`	validate the compiled graph
`show <f.sop.toml>`	render the compiled topology
`run <f> --db P`	execute; `--run-id`, `--input`, `--allow-write`
`resume <id> --db P`	continue a suspended SOP run

```
pii graph lint graphs/queue.toml                # static, offline, no store written
pii graph show graphs/night.toml                # the loaded topology, after weaving
pii run start night --input '{"workdir":"/projects/pi-improved","to":"status"}'
pii run get r000000000261                       # header + coder lines
pii run trace r000000000261 --reason implement  # why the node ended there
pii run resume r000000000261                    # continue after a gate
```

5.3 The queue

The queue is a store of work items, one file beside the main store (`queue.redb`, tables `queue_item` and `queue_event`). Each verb is one RPC; none opens the file.

Verb	Flags and effect
`add`	`--id`, `--spec`, `--title`, `--criteria` (repeatable), `--touches` (repeatable). Writes one pending item.
`ls`	every row with its status and reason
`claim --by <name>`	claim the next pending item for an external worker

An item is `pending`, `running`, `done` or `parked`. The graph's `select` node claims one and writes `running`; the `record` node writes `done` when the verdict is a pass or the literal `done`, and `parked` with the failing criterion otherwise. A row that stays `running` is therefore an operator question, not a bug in the queue: either the run is still alive, or it died holding the claim.

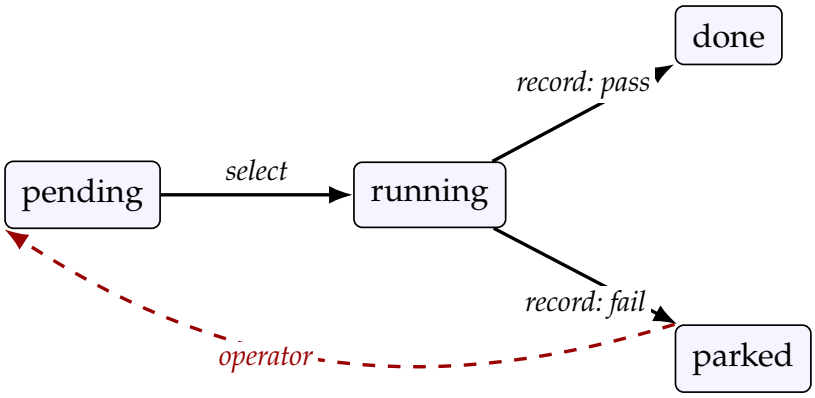


Figure 5.4: Four states, two writers. Only `select` and `record` move a row; the way back from `parked` is a person, not a retry loop.

5.4 Stores, backups and memory

`pii store`

Verb	Effect
`compact [--dir D]`	offline compaction; prints each file's measured size before and after. A store with nothing to reclaim prints a near-equal pair, which is the truth.
`stats [--dir D]`	one line per table with its row count, then the file's bytes
`prune [--dir D]`	one retention sweep under the default policy, then the count. Finished runs older than 30 days go, node events older than 7 days go first, at most 1,000 rows per sweep. A `Running` or `Suspended` run is never touched.

`compact` and `prune` open the store directly and therefore need the daemon stopped. `stats` does too, for the same reason; run it against a copy if the daemon must stay up.

`pii backup` and `pii restore`

Both are offline for the same reason, and both print a manifest line per file with its byte count, format version and blake3 digest.

```
pii backup /var/backups/pii-$(date +%F) --dir /var/lib/pii
pii restore /var/backups/pii-2026-11-04 --dir /var/lib/pii --force
```

`restore` refuses to overwrite a non-empty target unless `--force` is given.

`pii memory`

The memory family is the widest in the binary, and its subcommands split into three kinds: text search, the entity graph, and lifecycle.

Verb	Flags and effect
`search <q>`	`--k N` (default 6). BM25 over the document table; no model.
`put <text>`	`--layer L` (default `l1`), `--kind K` (default `atom`). One document, written once.
`learn`	`--run`, `--status`, `--reason` (required); `--graph`, `--attempts`, `--session`, `--commit`. Writes an outcome as an `atom`; the key is run+status+reason, so a retried report is `unchanged`.
`consolidate`	fold recent atoms into longer documents
`entity add`	`--kind`, `--name`, `--source` (repeatable, `run:/file:/actor:`), `--attr k=v`
`entity show <id>`	one record, its tier, and why it holds that tier
`entity list`	`--kind K` or `--name N`; `--limit`, `--rank`
`link <from> <to>`	`--kind` or `--rel`, `--source`, `--attr`
`neighbours <id>`	`--depth` (default 1, capped by config), `--limit`
`compact`	compact `memory.redb`; daemon stopped
`show <id>`	one document with its tier and reason
`tiers`	counts per tier, and the policy in force
`archive`	`--before <epoch|Ns|Nm|Nh|Nd ago>`; writes `<dir>/memory/archive-<ts>.jsonl`
`restore`	bring archived documents back into the live tables

Every graph bound is also a flag, so a one-off query can be narrowed without editing the config: `--max-entities`, `--max-relationships`, `--max-provenance`, `--max-depth`, `--max-results`, `--max-bytes`. The tier policy has `--hot-secs`, `--warm-secs` and `--hot-count`. A bound that is exceeded is refused by name (`entity_cap`, `result_cap`, `bytes_cap`), never truncated silently.

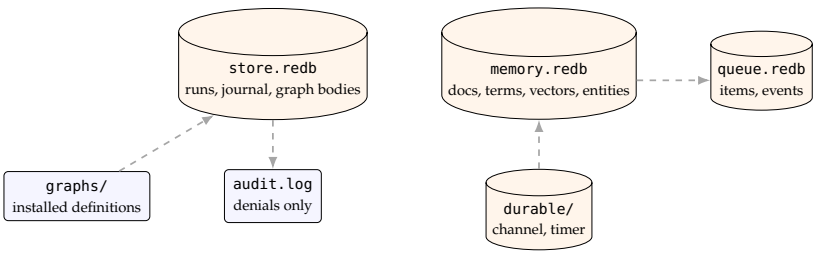


Figure 5.5: What an installation actually holds. The main store, the memory store, the queue and the durable area are separate redb files; the audit trail is a text file because it must remain readable when a store is not.

5.5 Knowledge, skills and indices

Verb	Flags and effect
`learn scan`	`--traj`, `--min-runs`, `--outlier-factor`, `--max-files`, `--max-file-bytes`. Reads trajectories, proposes candidate lessons.
`learn list`, `learn show <id>`	the candidate table, one candidate
`learn verify`	`<id>` or `--all`; `--max-runs` (5), `--run-ms` (30000), `--scratch`, `--protect` (repeatable), `--input`. Re-runs the evidence in a scratch tree.
`learn promote`	requires `--metric graph_definition_changed` and `--window-secs`; `--kind quarantine_graph`, `--ttl-secs`, `--actor`
`learn promotions`	`--sweep` to expire lapsed promotions
`learn revert <id>`	take a promotion back, with an actor and a ledger row
`learn ledger`	`--tail N`; the append-only record of every promotion
`skill add`	`--file`, `--layer L1|L2|L3`. The pinned script is hashed where it lies and never executed.
`skill list`	`--all`, `--layer`
`skill show`, `skill explain`	one skill; `explain` renders the `WHEN TO USE`, `CONTRACT` and `VERBS` sections verbatim
`skill edit <id>`	`--file`, `--note`; records the commit or file and time as evidence
`index code`	`--root`, `--max`, `--dir`, `--no-docs`; skips `target`, `_tmp` and `vendor`
`index docs`	`--root`, `--kind spec|handoff|ops|all`; one document per spec, per `HANDOFF` section, per operations doc

`index docs` is idempotent by construction: a document's id is the hash of its text, so a second pass over unchanged files reports `unchanged` and writes nothing.

5.6 Observation and decisions

Verb	Flags and effect
`health`	`--json`. Exit 0 when the daemon answers from this client's revision, 2 when nothing answers, 3 on drift or an unprovable comparison.
`metrics`	the gauge and counter text format, one line per series
`sys status`, `sys reload`	the status payload; re-read the config and the project layer without a restart
`disk`	`--root <repo>`; artefact roots with bytes, age, filesystem, then free space for `/` and for the repository's volume. Read-only.
`incidents scan`	`--root`, `--new`; or `--source runtime --log F` for the daemon's own log, with `--ingest --dir D` to store what it found
`incidents show`	`--dir`, `--id`
`incidents distance`	`--since YYYY-MM-DD`
`loop emit`	`--cycle`, `--kind`, `--detail`, `--file`. Always records locally; exit 3 if the daemon did not answer, so the caller can record the gap.
`loop events`	`--last N` (20), `--file`, `--log`
`cron ls`	configured cron entries
`approve ls`	pending approvals
`approve allow|deny <id>`	`--note`; resolves one gate
`heal ls`, `heal show <sig>`	the heal-attempt ledger, one signature
`net ls`	the listeners the daemon reports
`worker ls`, `worker reload <name>`	supervised children; drain and reload
`treat export`	`--out <dir>`; writes `manifest.json`, `fix.patch`, `repro.sh`
`treat import`, `apply`, `list`	`--check`, `--is-ancestor`, `--yes`
`deploy-card render`	`--card F`; refuses a card missing any of the five fields, naming the field
`deploy-card approve`	`--card F --rev R`; bound to the revision the card names
`deploy-card refuse`	`--card F [--reason T]`

Every deploy-card decision appends one line to the operator acts file, whose default is `_tmp/autonomy/operator-acts.log` and whose path is overridable with `$PII_ACTS`. That file is the evidence the distance metric counts; a decision that is not in it did not happen.

5.7 Devices, sandboxes and scaffolding

Verb	Flags and effect
`mqtt ls`	devices, from their own discovery messages
`mqtt sub <filter> [secs]`	what a filter carries, for a bounded window
`mqtt pub <topic> <text>`	`--qos 0|1|2`, `--retain`
`mqtt send <device> <cmd>`	address a device by name, not by topic
`sandbox-exec`	`--profile <p> --workspace <d> -- <argv>`; applies Landlock, verifies enforcement, then execs
`worker-exec`	the reference worker on stdio; ops `echo`, `sleep`, `crash`, `info`
`tui`	`--connect auto|loopback|<socket>`, `--sandbox`, `--workspace`. Never auto-starts a daemon.
`project init <dir>`	writes `.pii/settings.toml`, `.pii/policy.toml`, `.pi/`, `.claude/` and five `.gitignore` lines; refuses a target that already holds a credential
`lane plan <N>`	`--resume <sid>`, `--dry-run`; prints the agent, model, provider, spec, brief, log and wait, and writes nothing when dry
`lane run <N>`	the same plan, then the run
`lane rev`	the revision the lane binary was built from
`soak`	`--connect <socket> --runs N --graph G [--concurrency C]; N runs against a live daemon`

`mqtt` reads the broker from `--url`, else from `$PII_MQTT_URL`, and otherwise uses the loopback broker on port 1883. Its output is one JSON object per call — the same shape the model's tool returns — so a script and an agent see the same thing.

5.8 Exit codes

The binary uses a small, stable set. Scripts depend on these, and a change to one is a breaking change.

Code	Meaning
0	success
2	usage error, a refused control call, or an unreadable store. The message names the argument or the call.
3	the work ran and did not pass: graph validation errors, `health` drift or an unprovable revision comparison, `loop emit` with the daemon down
4	the sandbox profile could not be enforced; the daemon refuses to start unconfined
8	the lane's dirty-tree stop; the tree has changes the run did not make

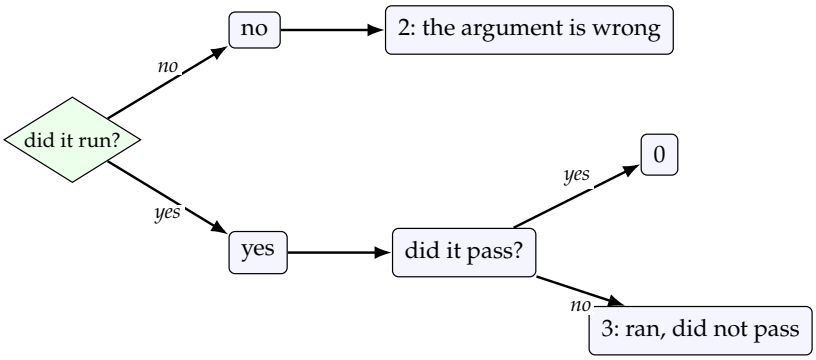


Figure 5.6: The decision an exit code encodes. “Ran and did not pass” is deliberately distinct from “did not run”: a script that retries on 3 and reports on 2 is doing different things, and should say so.

```
pii health >/dev/null; echo $?      # 0 same build, 3 drift, 2 no daemon
pii graph lint graphs/night.toml    # 0 ok, 3 invalid, 2 unreadable
pii store stats --dir /var/lib/pii  # needs the daemon stopped
pii metrics | grep pii_store_bytes  # one series per store file
```

6 Configuration

This chapter answers what the daemon reads at start, which of two files wins when both set the same key, what it refuses to read at all, and how to change a setting without a restart. It also states where every file an installation writes actually lives, because the most common configuration fault is not a wrong value — it is a second daemon reading different files from the one the operator is looking at.

6.1 Two files, one effective config

There is a global file and, optionally, a project file. The global file is the one an operator edits for a host; the project file is the one a repository carries, and it may set only the settings a repository has any business setting.



Figure 6.1: Resolution order. The first of flag, `$PII_CONFIG` and the default path decides which global file is read; the project file is then layered over it, and every unset key falls through to a built-in default.

The default path is `$HOME/.pii/pii.toml`, and it is resolved *before* the runtime directory. That ordering is deliberate: `[core].runtime_dir` lives in the file that sets it, so a config path derived from the runtime directory would be circular.

A file that does not exist is not an error at the default path: the daemon starts with built-in defaults and then refuses on the missing model. The same is true of a `--config` path that does not exist, which is worth knowing: a typo in the flag does not say "no such file", it says "no model configured" a moment later.

6.2 The project layer, and what it may not say

A workspace may carry `<workspace>/.pii/pii.toml`. It is read at start and on `pii sys reload`, through one function, so the two can never disagree about which file is read. A project file that exists but does not parse is fatal; it never silently falls back to the global file.

The merge is not a general merge. A project value wins for three things only:

Project may set	Why it is safe
`[core].model`	the model a repository's work should use
`[llama].host`	where that model is served, per repository
`[[model_tier]]`	the routing table, replaced whole rather than merged

Everything else that decides where the daemon writes or what it may reach is refused, and the refusal names the setting:

```
error: project config sets global-only settings: [net], grant_ceiling,
[core].runtime_dir (only the global ~/.pii/pii.toml may set them)
```

The list is short and closed: `[net]`, `grant_ceiling`, `[core].runtime_dir`, `[core].log_file`, `[core].audit` and `[core].observer`. The reason is asymmetric risk. A repository that could widen `[net] grants` would let a checkout grant itself egress; a repository that could move `runtime_dir` would point a daemon at a store nobody is watching. A model id, by contrast, is exactly the kind of thing that should differ per project.

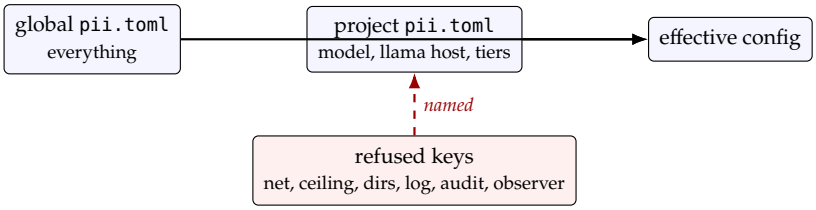


Figure 6.2: The layer is a whitelist, not a patch. A project file is a set of opinions about models; anything that decides where bytes land belongs to the host.

6.3 The file is strict

Every section is deserialised with unknown fields denied. A misspelled key is a load error that names the key, not a setting that silently does nothing. This is a security property rather than a style: `sandbx = "workspace"` must not start an unconfined daemon.

Two refusals are worth their own paragraph because they have been hit in practice:

- `[core].ollama_host` is refused by name. The ollama provider was retired; the key is not merely unknown, it produces a message that says to set `[llama] host` instead.
- Two `[[model_tier]]` entries with the same name, or an entry with an empty name or model, fail validation before the daemon binds a socket.

6.4 The sections

Key	What it decides
`grant_ceiling`	capability names no graph or node may request beyond; empty means no ceiling. An unknown capability name is an error. This is the one setting a graph author cannot argue with.
`[core]`	the host's own settings: model, directories, log, switches
`[llama]`	where `llamacpp-server:` models dial; default `http://127.0.0.1:8080/v1`
`[net]`	egress policy: `grants` (host:port patterns) and `allow_private`. Default: no grants at all.
`[[worker]]`	a supervised child: `name`, `program`, `args`, `env`, `max_in_flight`
`[[channel]]`	an inbound message source and the graph it starts per message
`[[mqtt]]`	one broker connection and the graph each message starts
`[hal]`	hardware backend (`sim` or `gpiod`) and the pins, I2C and SPI a graph may claim. Absent = the HAL is off and a `hal` node fails.
`[wasm]`	plugin host: the one directory modules load from, plus per-call `timeout_ms`, `max_memory_mib` and `fuel`. Absent = off.
`[memory]`	`path`; empty means the store beside the main store
`[memory_graph]`	the bounds on the entity graph and the tier policy
`[[cron]]`	a named timer entry: `name`, `graph`, `schedule`, `input`
`[[model_tier]]`	a named model: `name`, `model`, `tools`, `max_tokens`
`[evolve.routing]`	step to model, for the steps `localize`, `patch`, `review`, `research`, `orchestrator`. A typo'd step is an error, not a silent no-route.

`[core]`

Key	Meaning	Default
`model`	the model spec runs use when a graph names none	none
`runtime_dir`	sockets, installed graphs, crashes	per-uid
`durable_dir`	channel, timer and other durable stores	state/durable
`session_runs_cap`	runs remembered per session in memory	built-in cap
`trajectory_dir`	when set, each run appends a JSONL trajectory	off
`trajectory_max_bytes`	aggregate bound on the trajectory directory	512 MiB
`trajectory_max_files`	aggregate file bound	50,000

Key	Meaning	Default
`learn_interval_secs`	the unattended learn pass interval	86,400
`log_file`	daemon events as JSON lines	off
`log_rotate_bytes`	rotate before a line passes this size	10,485,760
`log_keep_files`	rotated files kept beside the live one	5
`observer`	`on`/`off`; measurement only	`on`
`audit`	`on`/`off`; measurement only	`on`

Three keys read a zero in a way that is easy to guess wrong, so they are stated here rather than left to the sample. `learn_interval_secs = 0` switches the unattended learn pass off; the field is optional and its absence means every 86,400 seconds. `trajectory_max_bytes` and `trajectory_max_files` treat a zero as "unset" and fall back to the defaults, never as "unbounded", because the trajectory directory is always bounded. `log_rotate_bytes = 0` is different again: it is passed through, and the sample documents it as "never rotate" — which is a legitimate choice for a log a supervisor already rotates, and a bad one for a host nobody watches.

The two measurement switches deserve a warning that the file cannot give: switching `audit` off stops denials reaching `audit.log`, and switching `observer` off detaches tool-call, store and provider logging. Both announce themselves as warning events at start, and both are documented as measurement-only. An installation that runs with either off is not observably the same installation as one that runs with them on.

`[memory_graph]`

These bounds are also command-line flags on `pii memory`, so a one-off query can be narrowed without touching the file.

Key	Default	Refusal when exceeded
`max_entities`	10,000	`entity_cap`
`max_relationships`	50,000	`relationship_cap`
`max_provenance`	32	`provenance_cap`
`max_attributes`	32	none; the extra attribute is refused
`max_depth`	3	depth clamp at the call
`default_depth`	1	—
`max_results`	200	`result_cap`
`hot_secs`	86,400	tier policy
`warm_secs`	604,800	tier policy
`hot_count`	10	tier policy
`max_bytes`	16 MiB	`bytes_cap`

6.5 Where the files are

Three directories, and they are not interchangeable. The state directory holds what must survive; the runtime directory holds what a restart may

rebuild; the workspace is what the sandbox confines writes to.

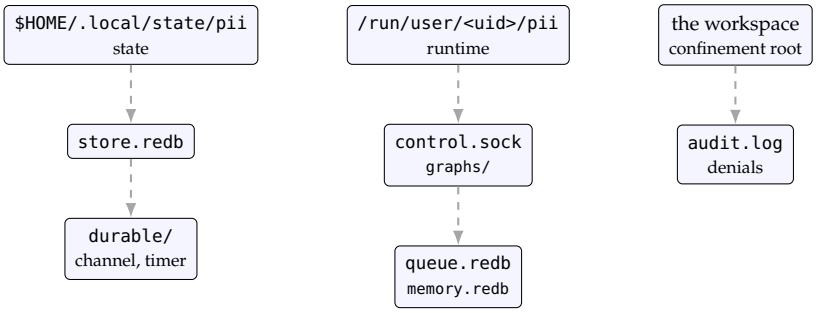


Figure 6.3: The layout an operator has to hold in mind. The audit trail lives beside the store, not in the log directory, so it survives losing a log volume.

Path	Contents and note
`$PII_STATE_DIR`	the state directory; default `$HOME/.local/state/pii`
`$PII_RUNTIME_DIR`	sockets and ephemeral files; default `/run/user/<uid>/pii`
`store.redb`	the main store, in the runtime or state dir; `--store` overrides
`audit.log`	beside the store: denials and the sandbox profile in force, rotated like the log
`queue.redb`	beside the store: the queue's items and events
`memory.redb`	beside the store: documents and the entity graph
`durable/channel.redb`	under the state dir: channel cursors and last started runs
`durable/timer.redb`	under the state dir: the timer wheel
`graphs/`	under the runtime dir: installed definitions, `<name>.toml`
`./graphs`	the repository copy that `graph list` compares against

`pii graph list` prints `DRIFT` when the installed copy differs from the repository copy, which is the fastest way to answer "is the daemon running what I reviewed".

6.6 Secrets are not configuration

No credential is a config value. The file names the *variable* or the *file* that holds a secret, never the secret:

- `[[channel]]` uses `token_env` or `token_file`, and `secret_env` or `secret_file` for webhook signatures.
- `[[mqtt]]` uses `username_env` or `username_file`, and `password_env` or `password_file`. Credentials in clear text to a non-loopback broker are refused unless `allow_plaintext_credentials` is true.
- Provider keys come from `OPENAI_API_KEY`, `ANTHROPIC_API_KEY` or `GEMINI_API_KEY` in the daemon's environment, and are scrubbed from provider errors and events.

The reason is not tidiness. A config file is copied into backups, printed in bug reports and mounted into containers; an environment variable is not. The scrubber that removes secret-named variables from log lines is built from the daemon's own environment, so a key passed as a `--config` value would be invisible to it.

6.7 Environment variables

Variable	Effect
`PII_CONFIG`	the config file, overriding the default path
`PII_STATE_DIR`	the state directory
`PII_RUNTIME_DIR`	the runtime directory; wins over `[core].runtime_dir`
`PII_MODEL`	the model when `[core].model` is unset; the flag or the file wins
`PII_TRAJECTORY_DIR`	trajectory output; overrides `[core].trajectory_dir`
`PII_MQTT_URL`	broker for `pii mqtt`, after `--url`
`PII_ACTS`	the operator acts file the deploy-card and distance commands append to
`PII_LOOP_EVENTS`	the loop event NDJSON file
`PII_AGENT`	the lane's agent: `claude` or `pi`
`PII_GRAPH_EDIT_OK`	permits graph edits from the spec lane
`ALLOW_DIRTY`	permits the lane to start on a dirty tree, for a resume
`OPENAI_BASE_URL`	an OpenAI-compatible endpoint; any vendor with that shape
`OPENAI_API_KEY`	its key; optional for a local server
`ANTHROPIC_BASE_URL`	default `https://api.anthropic.com`
`ANTHROPIC_API_KEY`	required for Anthropic models
`GEMINI_BASE_URL`	the Google endpoint; the default is the public host
`GEMINI_API_KEY`	sent as `x-goog-api-key`, never in the URL

6.8 Changing a setting

There are three cost levels, and choosing the wrong one is how a host ends up running a config nobody edited.

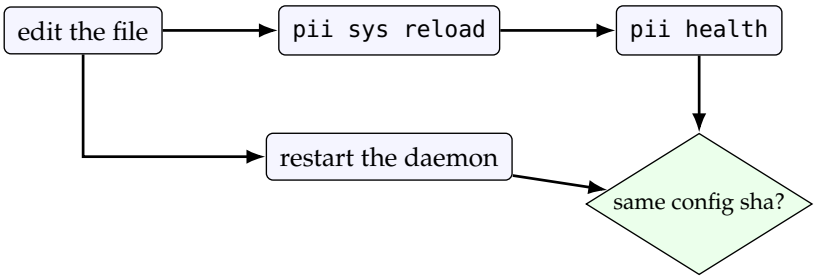


Figure 6.4: A change is not in force until a surface says so. `pii health` prints the config path and a sha256 of the file it read, which is the only proof that the daemon is running the file you edited.

`pii health` prints, on its first three lines, the binary's version, target and revision, this client's revision, and the config path with its digest:

```
pii 0.2.0 (4249d5b2fa85, x86_64-unknown-linux-gnu)
client: 4249d5b2fa85
config: /root/.pii/pii.toml sha256=9f2c...
```

If the digest is not the file you just saved, the daemon read a different file: check `$PII_CONFIG` in the service unit before editing anything else.

`pii sys reload` is narrower than its name suggests, and knowing its exact scope saves an afternoon. It re-reads the global file and the project layer, then applies exactly two things: every configured worker is reloaded, and each running channel's graph name is replaced. It returns the number of workers reloaded. Nothing else changes — not the model, not the memory bounds, not `[net] grants`, and not a channel's `poll_interval_ms`, which is read once when the poll loop starts. For anything outside those two, the daemon has to be restarted.

Setting	`sys reload`	Restart
`[[worker]]` entries	applied	applied
`[[channel]].graph`	applied	applied
`[[channel]].poll_interval_ms`	no	applied
`[core].model`, `[llama].host`	no	applied
`[net].grants`, `grant_ceiling`	no	applied
`[memory_graph]` bounds	no	applied
`[core].runtime_dir`, `log_file`	no	applied at next start

The boundary has one practical consequence worth stating: a policy change — egress grants, the capability ceiling, the model — is never in force until a restart, so a host that has been edited but not restarted is running a config nobody can see from the file.

6.9 A production file, in full

The repository ships `examples/pii.production.toml`, which lists every key with its default. The parts an administrator changes first:

```toml
[core]
model = "llamacpp-server:granite-4.2-3b"
runtime_dir = "/run/pii"
log_file = "/var/log/pii/pii.log"
log_rotate_bytes = 10485760
log_keep_files = 5

[llama]
host = "http://10.10.10.217:8082/v1"

[net]
grants = ["127.0.0.1:11434"]
allow_private = true

[memory]
path = ""

[[cron]]
name = "nightly"
graph = "night"
schedule = "0 3 * * *"
```

Two of those deserve comment. `[net] grants` is empty by default, so the file above opens exactly one host:port pair and nothing else; a model served elsewhere needs its own grant or every call is denied with `net.egress.denied`. And `schedule` is display-only: the timer that actually fires is the graph's entry node, so a cron entry whose graph does not start with a `timer` does not run on a clock at all.

6.10 Refusals, and what they look like

Symptom	Cause and check
`config: unknown field ...`	a misspelled key. The message names it; the file is strict on purpose.
`[core].ollama_host is retired`	the ollama provider is gone; use `[llama] host` with `llamacpp-server:<model>`
`project config sets global-only settings`	the repository tried to set `[net]`, `grant_ceiling`, a directory, the log or a switch
`duplicate [[model_tier]] name`	two tiers with one name; the routing table must be unambiguous
`no model configured` (exit 5)	neither `[core].model` nor `$PII_MODEL`; also what a typo'd `--config` path looks like
exit 4 at start	the requested sandbox profile could not be enforced; the daemon refuses rather than run unconfined
a setting that has no effect	the daemon read another file. Compare the `config: ...sha256=` line in `pii health` with the file on disk.

```
pii daemon --config /etc/pii/pii.toml --workspace /projects/pi-improved \
           --sandbox workspace --log-file /var/log/pii/pii.log
pii health --json | jq '.config, .mounts'    # what it read, what it may
    ↪ touch
pii sys reload                               # re-read the file, no
    ↪ restart
```

7 Observability

An operator asks three questions of a running pii, and each has one surface that answers it faster than the others: *is it up and is it the build I think it is*, *what did this run actually do*, and *why was that refused*. This chapter covers those surfaces in that order — the daemon's account of itself, the event stream, the metric set, the run journal, and the audit trail — and ends with the correlation order that turns five commands into one answer.

The figures and tables are generated from the code: the event names from `crates/pii-observe/src/names.rs`, the metric names from the same file, the journal shape from `crates/pii-abi/src/journal.rs`, and the command output from `crates/pii-cli/src/cmd_health.rs` and `cmd_metrics.rs`.

7.1 Four surfaces, four questions

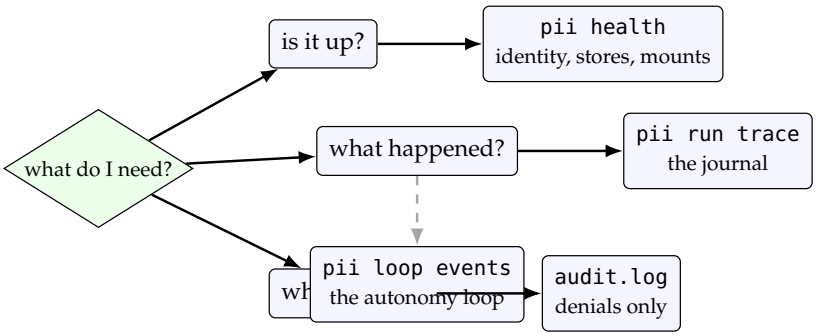


Figure 7.1: The entry point for each question. `pii health` is the right first command for all three, because it tells you whether the other surfaces can answer at all.

Surface	Written by	Good for, and its limit
`pii health`	the daemon, on request	identity, config digest, stores, mounts, counters since deploy. It is a snapshot, not a history.
the event log	the daemon, per event	everything the daemon decided. Off unless `--log-file` or `[core].log_file` is set.
`audit.log`	the daemon, per denial	the security-relevant subset. Always on unless `--audit off`. Never contains a kernel-level denial raised inside a child.
the run journal	the engine, per run	what a run did, node by node, replayable. Grows with runs; retention applies.

7.2 The anatomy of one event

Every event is one JSON object on one line. The file sink adds a wall-clock stamp; the rest of the object is the event itself.

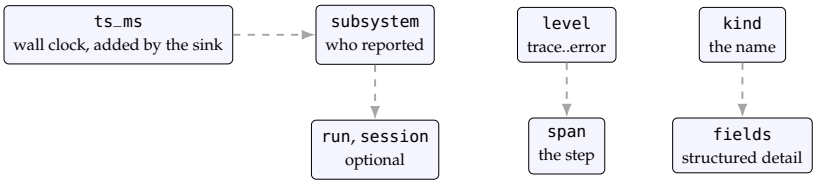


Figure 7.2: Six fields are always present (`ts_ms`, `subsystem`, `level`, `kind`, `fields`, `span`); `run` and `session` appear only when the event belongs to one.

The serialised form, with the run and session fields omitted because the event has neither:

```
{"subsystem":"x","level":"info","kind":"k","fields":{"a":1},"span":null}
```

`level` is one of `trace`, `debug`, `info`, `warn`, `error`. In practice the daemon emits `info` for lifecycle, `warn` for a refusal and `error` for a failure it could not prevent; `trace` and `debug` exist in the type and are not the ones to grep for during an incident.

The two correlation fields are the reason a line can be trusted. `run` is the run id and `session` the session id, so one `grep` answers "everything that happened to this run" without matching a node name that three graphs share.

7.3 The name registry

Names are not string literals at call sites. Every event and every metric is a constant in one file, and the file also holds an `ALL` array that a unit test checks for duplicates and empty names. The count is exact and worth knowing, because it is the size of the vocabulary an operator has to recognise:

Kind of name	Count	Where it is used
event names	150	the `kind` field of a log line, and the audit filter
metric names	40	the series names `pii metrics` prints
the `ALL` registry	1	the duplicate check, not emitted
constants in the file	191	150 + 40 + 1

The 150 events fall into 38 subsystems. The largest are the ones an installation touches every day: channels, learning, models, memory, runs.

Subsystem	n	Subsystem	n	Subsystem	n
channel	14	model	11	daemon	7
learn	13	memory	10	mqtt	6
run	10	coder	9	hal	6
wasm	8	worker	5	trajectory	4
proto	4	sop	4	graph	3
net	3	cron	3	evolve	3
timer	3	tools	2	queue	2
tui	2	web	2	fetch	2
effect	1	store	1	core	1
health	1	observe	1	criteria	1
skill	1	cli	1	lint	1
cursor	1	mcp	1	node	1
log	1	uds	1		

A few of the names are load-bearing enough to memorise, because they name the failures that are otherwise silent:

Name	What it means when you see it
graph.node.dead	a node was marked dead; the run cannot proceed through it
graph.join.unsatisfiable	a join can never be satisfied; the graph is wrong, not the run
observe.events.dropped	the event pipeline dropped events; your log has a gap
core.cap.denied	a capability the node did not hold was requested
store.persist.error	a write to a store failed; the run's account is incomplete
model.provider.auth_-failed	the key, not the model, is the problem
model.provider.rate_-limited	the provider refused for quota; retrying now is waste
coder.env.refused	a variable named in env_pass was not passed

7.4 Metrics

`pii metrics` asks the daemon for its status payload and renders gauges and counters in the text exposition format, one series per line, with labels in braces:

```
pii_store_bytes{store="main"} 1048576
pii_runs 42
pii_run_latency_ms{outcome="succeeded",quantile="p95"} 132
```

It needs a running daemon: with none, it prints "no daemon answered" and exits 2. The same numbers are in `pii health --json`, which is the better choice when you want to keep the payload for a record rather than read it.

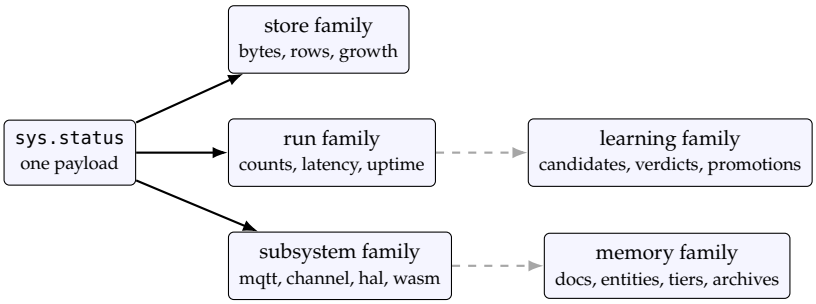


Figure 7.3: Six families, one payload. Every metric is derived from state the daemon already holds, so asking for metrics never changes what it is doing.

The full set, with the label each series carries. *Gauge* means a value that can go down; *counter* means one that only grows until the daemon restarts.

Series	Type	Meaning
`pii_runs`	gauge	runs in the main store
`pii_sessions`	gauge	sessions in the main store
`pii_approvals_pending`	gauge	approvals awaiting a decision
`pii_journal_events_total`	counter	journal events ever appended
`pii_memory_docs`	gauge	memory documents
`pii_cron_entries`	gauge	configured cron entries
`pii_store_bytes`	gauge, `store`	size of a store file in bytes
`pii_store_format_version`	gauge, `store`	persisted format version
`pii_rss_bytes_per_run`	gauge	RSS growth per run over the recent window
`pii_process_rss_kb`	gauge	resident set size in KiB
`pii_store_bytes_per_run`	gauge	main-store bytes divided by runs
`pii_run_latency_ms`	gauge, `outcome`, `quantile`	run latency, upper bucket bound
`pii_run_latency_count`	counter, `outcome`	runs in the histogram
`pii_uptime_ms`	gauge	daemon uptime
`pii_mqtt_received_total`	counter	messages accepted from brokers
`pii_mqtt_dropped_total`	counter	messages dropped: queue full, not UTF-8
`pii_mqtt_published_total`	counter	messages published
`pii_worker_requests_refused_total`	counter	refusals around a drain or reload
`pii_channel_posts_total`	counter	messages a `post` node delivered
`pii_channel_post_refusals_total`	counter	`post` calls refused or failed
`pii_quarantine_refusals_total`	counter	runs refused by a promotion quarantine
`pii_quarantine_refusals_recorded_total`	counter	refusals whose deny reached the ledger; a gap to the line above is a finding
`pii_hal_ops_total`	counter, `kind`	HAL operations that succeeded
`pii_hal_refusals_total`	counter	HAL operations refused by claim or scope
`pii_wasm_calls_total`	counter, `outcome`	wasm calls by outcome
`pii_wasm_refusals_total`	counter	modules or calls refused before running
`pii_learn_candidates`	gauge, `kind`	candidates from the last scan
`pii_learn_refusals_total`	counter	candidates refused for missing evidence
`pii_learn_candidates_proposed_total`	counter	distinct candidates the unattended loop proposed
`pii_learn_unattended_runs_total`	counter	unattended extract+verify passes
`pii_learn_verdicts`	gauge, `verdict`	verdicts of the last verification

Series	Type	Meaning
pii_learn_promotions	gauge, state	promotions by state, plus refused
pii_memory_entities	gauge	entities in the memory graph
pii_memory_relationships	gauge	relationships in the memory graph
pii_memory_writes_total	counter, kind	writes that changed the graph
pii_memory_write_refusals_total	counter, reason	memory writes refused
pii_memory_query_refusals_total	counter, reason	memory queries refused
pii_memory_tier	gauge, tier	records in each tier
pii_memory_tier_changes_total	counter, change	transitions, <from>_to_<to>
pii_memory_archive_ops_total	counter, op	archived, restored, refused

Two of these are instruments rather than descriptions. `pii_store_bytes_per_run` is the growth rate as one number: a store that is not growing prints a small value, and a value that climbs after every run names the subsystem to look at. `pii_quarantine_refusals_recorded_total` exists to be compared with `pii_quarantine_refusals_total`: a refusal that did not reach the ledger is a defect in the refusal path, not in the run.

7.5 Health

`pii health` prints the identity-bearing parts of the status payload, in a fixed order:

```
# the shape of the output; values abbreviated
pii 0.2.0 (4249d5b2fa85, x86_64-unknown-linux-gnu)
client: 4249d5b2fa85
config: /root/.pii/pii.toml sha256=9f2c...
stores:
  main: /run/user/0/pii/store.redb (1048576 bytes, format_version=1)
mounts:
  workspace [rw]: /projects/pi-improved  (workspace)
since_deploy:
  revision: 4249d5b2fa85
  runs: 42
  starts: 3
  hard_deaths: 1
  runs_by_status.succeeded: 39
counts:
  runs: 42
```

Read it in three passes. *Identity:* the binary's revision and this client's must match; `config` names the file that is actually in force, with its digest. *State:* the stores and their format versions, then the mounts, which are the sandbox's answer to "what may this daemon write". *History:* `since_deploy` is the run of numbers that survives a restart — `starts` against `hard_deaths` is the crash record, and a `hard_deaths` that grew since yesterday is the first thing to explain.

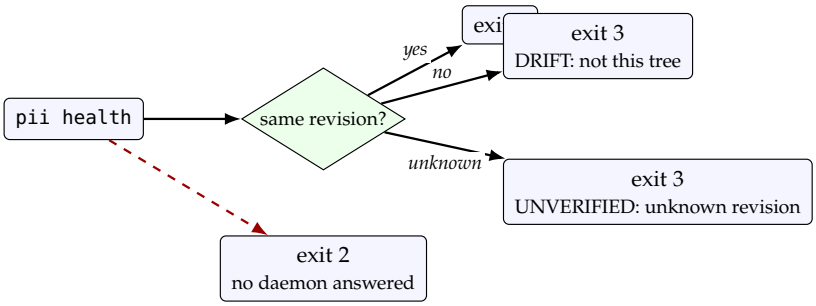


Figure 7.4: Three outcomes, two of them exit 3 for different reasons. "Unknown" is never treated as agreement: a build that cannot prove what it is has not proven it is current.

The distinction between drift and unknown matters in an upgrade. Drift means the daemon is a different, known binary: restart it with the new one. Unknown means one side has no revision stamped, which is what a locally built binary without the build script looks like — the comparison cannot be made, and `pii health` will not pretend otherwise.

7.6 The run journal

The journal is the only account of what a run did, and it is append-only: every event is written once, under `(run, seq)`. Seventeen event variants exist; the ones an operator reads are these.

Journal event	What it records
`Started`	the graph name and a hash of the run input
`TickStarted`	the batch of ready nodes drained in one scheduling tick, and its width
`NodeStarted`	a node began, with its millisecond offset
`NodeFinished`	the port it took, the value it produced, the value's hash, the duration
`NodeFailed`	the error, whether it is retryable, and the attempt number
`NodeProgress`	a heartbeat: elapsed time and child CPU. A wedged node shows elapsed growing while CPU stays flat.
`NodeCancelled`	a node stopped because a person cancelled the run
`NodeDead`	a node marked dead; the run cannot pass it
`NodeInterrupted`	the node was cut short by a suspension or shutdown
`Suspended`, `Resumed`	the run parked on a resume key, and continued
`NodeRetried`	an explicit retry of one node
`EffectStarted`, `EffectConfirmed`	a side effect began, and was confirmed
`RunSpawned`	a child run was started, with its id
`Finished`	the terminal outcome

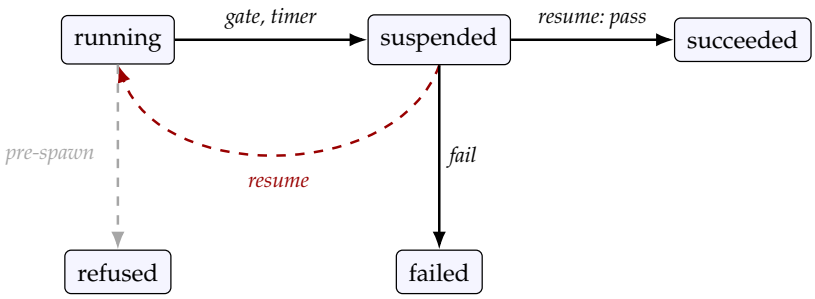


Figure 7.5: Five statuses, and the one that is easy to miss. `Refused` means the agent call was refused before anything was spawned: it is not a failure of the work, and it is not in the `run ls --status` vocabulary.

That last point is a real limitation of the filter. `run ls --status` accepts `running`, `suspended`, `succeeded` and `failed`; the store's fifth status, `refused`, cannot be named on the command line. A refused run therefore appears in an unfiltered listing and disappears from every filtered one, which is exactly when someone is looking for it.

```
pii run ls                       # refused runs appear only here
pii run trace r000000000261 --reason implement
pii run trace r000000000261 --overlapped code,docs
pii run trace r000000000261 --terminal
```

`--overlapped` is the query that answers a performance question without a profiler: two nodes that were supposed to be sequential and were not show up as overlapped, and two that were supposed to run together and never did show up as disjoint.

7.7 The audit trail

`audit.log` sits beside the store and receives only the security-relevant events. It is rotated like the log, it is on by default, and it exists so that an operator can answer "what was denied" without having enabled a full event log.

Kind in the trail	What it records
`core.cap.denied`	a capability the node did not hold
`tools.deny_list.hit`	a tool call matched the deny list
`fetch.allowlist.denied`	a fetch to a host outside the allowlist
`net.egress.denied`	egress outside `[net] grants`
`daemon.sandbox.in_force`	the profile, the exact grant, and whether confinement is applied
`daemon.peer.refused`	a control peer refused at the socket
`daemon.observer.detached`	the observer fan-out was switched off
`daemon.audit.disabled`	the audit feed was switched off

A start writes `daemon.sandbox.in_force` to the trail. It carries `profile`, `grant` and `enforced`, and it is the only place that states what the daemon was actually confined to rather than what it was asked for.

One limit is structural and stated in the module that implements the filter: a denial raised by the kernel inside a child process — a raw redirect blocked by Landlock — never passes through pii and is therefore not in the trail. Only denials pii itself decides appear.

```
grep daemon.sandbox.in_force /var/lib/pii/audit.log | tail -1
grep -c core.cap.denied /var/lib/pii/audit.log
grep net.egress.denied /var/lib/pii/audit.log | jq -r '.fields.host' |
    ↪ sort | uniq -c
```

With `--audit off` the file is not empty: it records exactly one kind, `daemon.audit.disabled`, so a trail that stops is distinguishable from a trail that was never written.

7.8 Incidents and the loop

Two surfaces cover the unattended side, and neither needs a model.

`pii incidents scan --source runtime --log F` reads the daemon's own log and reports each incident with its count, first and last seen, and `latency_secs` against the configured window — the detection latency, which is the number that says whether the alert arrived in time. Adding `--ingest --dir D` stores what it found; `pii incidents show` reads one back. `pii incidents distance --since` measures the operator's own response time from recorded timestamps.

`pii loop emit` writes one NDJSON line per lifecycle step of an autonomy loop, and `pii loop events` merges that file with the daemon log so the loop and the daemon read as one timeline. The exit code carries information the text does not: `emit` keeps the event locally and exits 3 when the daemon did not answer, appending `daemon=down` to the detail. A loop that records 3s is a loop that was running while the daemon was not.

7.9 Correlating a failure

The order matters, because each step narrows the next.

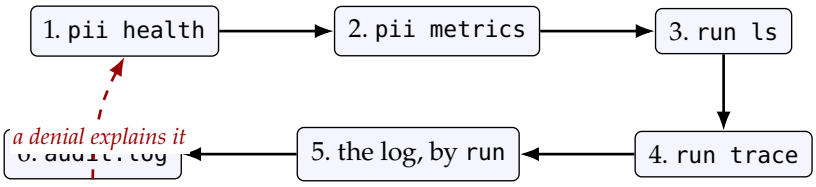


Figure 7.6: Six steps from "something is wrong" to a cause. Steps 1 and 2 rule out the daemon; steps 3 to 5 locate the run and the node; step 6 explains a refusal that the run itself could not.

Symptom	First surface, and what to look for
every run fails in a second	the log, for `model.provider.auth_failed` or `rate_limited`; the metric set carries no model series, so this is an event question, not a metric one
a run is "running" for hours	`run trace --node-status`, then `NodeProgress`: elapsed growing with flat CPU is a wedged child

Symptom	First surface, and what to look for
the run stopped with no error	`run trace --terminal`: a node left through a port with no edge, so nothing failed and nothing continued
a write was refused	`audit.log`: `core.cap.denied` or `tools.deny_list.hit`, never the run's own output
the log has a gap	`observe.events.dropped`: the pipeline dropped events, so absence is not evidence
the daemon restarted by itself	`pii health`: `hard_deaths` in `since_deploy`, and a log that ends on `daemon.started` with no `daemon.stopped`
the metrics do not move	the daemon serving them is not the one running the work: compare `pii_uptime_ms` with the process you restarted

```
pii health --json > /tmp/health.json
pii metrics | grep -E 'provider|store_bytes|latency'
pii run ls --status failed | jq -r '.[].run' | head
pii run trace "$RUN" --reason "$NODE"
grep "\"run\":\"$RUN\"" /var/log/pii/pii.log | jq -c 'select(.level!="info
    ↪ ")'
grep -E 'cap.denied|egress.denied' /var/lib/pii/audit.log | tail -20
```

8 The graph library

A graph is the unit of work in pii. An administrator needs three things from this chapter: which graph to run for a job, what each graph will do to the machine, and how to read a run that did not do what was expected. Every graph below was read from its file in `graphs/`; where the generated page in `graphs/DOCS/` disagrees with the file, the file wins and the disagreement is stated.

After the shape and the inventory, each graph gets one table with the same seven fields: purpose, nodes, ports, entry and exit, capabilities, when to run, and how it fails.

8.1 What a graph is, as an operator

A graph has an *entry* (where a run starts), named *ports* on every node (how a step reports its outcome), and — if another graph may use it — an *exit*. It declares a set of *capabilities*, and a node may hold only what its graph holds. A port is not a boolean: a judge has `ok`, `fail` and, in fail-expectation mode, `unrunnable`; a queue `select` has four outcomes.

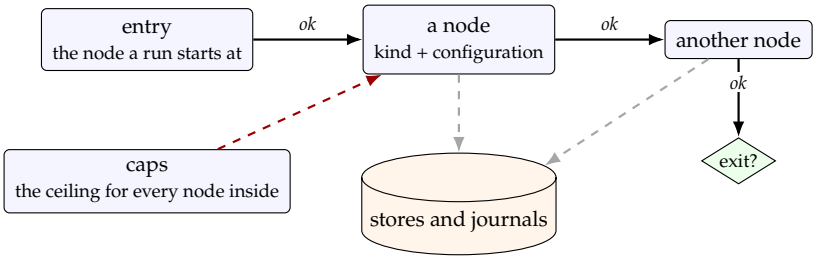


Figure 8.1: The five things to look at in any graph file: where it starts, what each node does, which ports it can leave by, what the graph may touch, and whether it declares an exit.

A graph without an exit is a *run*, not a component: `docs` can be called because it declares `exit = "docs"`, and `night` is callable because it declares `exit = "report"`.

8.2 The inventory

Thirty-one files live under `graphs/`. Twenty-seven have a generated page in `graphs/DOCS/`; the four that do not (`sdd`, `tdd`, `self-improve`, `doc-code`) are the most recently composed and the ones an operator is most likely to run.

Graph	Nodes	Entry → exit	Capabilities
`night`	4	`tick` → `report`	spawn, proc, post, fs_read, fs_write
`sdd`	8	`contract` → `done`	spawn, proc, fs_read, fs_write
`tdd`	9	`write_test` → `green`	spawn, proc, fs_read, fs_write

Graph	Nodes	Entry → exit	Capabilities
queue	3	select → record	spawn, proc, fs_read, fs_write
spec	7	code → done	spawn, proc, fs_read, fs_write
docs	1	docs → docs	spawn, proc
doc-code	8	inventory → done	spawn, proc, fs_read, fs_write
self-improve	6	case → report	spawn, proc, fs_read, fs_write
preflight	3	verify → —	proc
claude-probe	3	code → —	spawn, proc
coder-demo	3	code → —	spawn, proc
coder-podman	3	code → —	spawn, proc
coder-selffix	3	code → —	spawn, proc
code-index	4	— → —	— (does not load)
intervention-report	6	— → —	— (does not load)
digest-now	5	devices → —	memory_read, post
discord-digest	6	tick → —	memory_read, post
discord-triage	3	classify → —	post
podman-guard	2	— → —	— (fragment)
starter/assistant	4	agent → —	fs_read, fs_write, proc, net
starter/boss-watch	5	tick → —	proc
starter/consolidate	3	tick → —	—
starter/crash	1	boom → —	—
starter/failover	10	primary → —	fs_read, fs_write, proc
starter/heal-fix	11	probe → —	fs_read, fs_write, proc
starter/heal-watch	4	tick → —	proc
starter/react	3	agent → —	fs_read, fs_write, proc
starter/slow	2	wait → —	—
starter/soak	1	step → —	—
starter/soak_cron	2	tick → —	—
starter/verify-fix	5	fix → —	fs_read, fs_write, proc

Two files in that list do not load in this build, and that belongs at the top rather than in a footnote. `code-index.toml` and `intervention-report.toml` put their name in a `[graph]` table and express their edges as `depends_on` on each node; the loader reads a top-level `name`, an `entry`, and `[[edge]]` triples, so it refuses both with `missing 'name'`. They are design records, not runnable graphs, and `pii graph lint` says so in one line.

Three more files fail validation rather than loading: `tdd` and everything spliced from it. Section 8.3 has the measured status of all thirty-one files.

The generated pages are not stale by accident: they describe the *installed* copies. `pii graph list` compares each installed file with the repository copy and prints the verdict, and on the machine this chapter was written on four graphs were `DRIFT` (`queue`, `spec`, `coder-selffix`, `discord-digest`). The installed `queue.toml` was still the four-node `dispatch/judge` version dated 2026-11-03, while the repository held the three-node version that calls `spec`, dated 2026-11-04 — which is why `DOCS/queue.md` lists four nodes. `DOCS/claude-probe.md` records a budget of zero where the file says `max_budget_usd = 0.30`. When a page and a file disagree, ask which file the daemon loads: the installed one, and `graph list` is how you see it.

8.3 The lint status of the library

`pii graph lint` is the check that decides whether a file can run at all, so it was run against every file. The result is not what the file list suggests.

Exit	Files	What it means
0	26	loads and validates; warnings only (V5: a declared port with no edge — normal for a terminal node, and the thing to read for a node in the middle)
3	3	`tdd, sdd, night: V9: node '<n>' uses a multi-arm join with 1 inbound data edges, once per single-armed refuse node`
2	2	`code-index, intervention-report: invalid: missing 'name'`

The three V9 failures are one defect seen three times. Each of `tdd`'s refusal terminals (`not-red`, `unrunnable`, `still-red`) declares `join = "any"` and receives exactly one edge; the validator treats a multi-arm join with one arm as a typo, so it refuses the graph. `sdd` inherits the failure because it splices `tdd`, and `night` inherits it because it splices `sdd` — which is why the failure is reported with namespaced ids such as `sdd__tdd__not-red`.

The practical consequence is stark and belongs at the top of this chapter rather than in a footnote: *the TDD and SDD flows do not lint in this build, so they cannot start*. What does run today is `queue`, `spec` and `docs` (the row lane), `doc-code`, `self-improve`, `preflight` and the samples. An operator who schedules `night` will see it refuse at load, with the V9 lines printed to stderr and exit 3.

```
pii graph lint graphs/tdd.toml     # 3 x V9, exit 3
pii graph lint graphs/sdd.toml     # the same nodes, namespaced tdd__not-red
pii graph lint graphs/night.toml   # the same nodes, namespaced sdd__tdd__not-red
pii graph lint graphs/queue.toml   # exit 0, warnings about record's unused ports
```

8.4 How the pieces compose

Only a few `call` nodes exist, and each one is a decision that a callee is the right owner of a step.

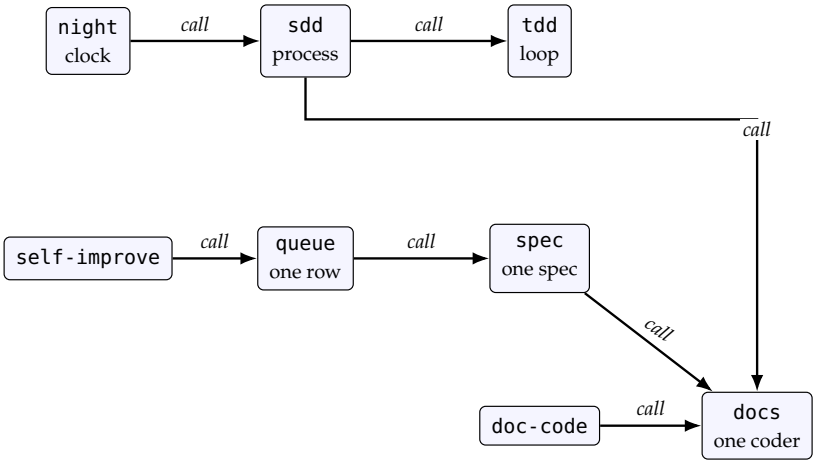


Figure 8.2: Every `call` in the library. Each arrow is pinned by sha256, so a change to the callee refuses to load until someone re-pins it deliberately.

The pinned digest is what makes composition safe rather than convenient: `sdd` pins `tdd` and `docs`, `queue` pins `spec`, and `night` pins `sdd`. Edit `docs.toml` and every host that calls it refuses to load; that refusal is the review gate.

8.5 The operator flows

`night` — the clock

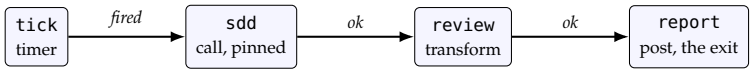


Figure 8.3: `night` adds only two things to the work: time, and telling a human. Everything between is a call.

purpose	run the day's work on a clock and report it; it implements nothing itself
nodes	`tick` timer; `sdd` call(`sdd`); `review` transform; `report` post
ports	`tick.fired`, `sdd.ok`, `review.ok`; `report` is terminal
entry/exit, caps	`tick` → `report`; `max_wall_ms = 7200000`; spawn, proc, post, fs_read, fs_write
when	a host that should work unattended overnight; the run input needs `workdir` and `to`
lint	exit 3: the spliced `tdd` refuse nodes fail V9, so the graph does not load in this build
fails	a changed `sdd` refuses to load (pin); a refusal inside `sdd` has no edge here, so the night stops silently; a missing `to` fails at `report`

The silent stop is the one to watch for, and it is a property of the splice rather than of `night`: `sdd`'s `unverified` and `refused` nodes are `refuse` nodes, which declare no `ok` port, so a refused implementation never returns a port the host can route. `queue` avoids this by wiring `implement.unverified` and `implement.refused` by name; `night` does not wire them, because until recently the callee had no such ports to name.

`queue` and `spec` — one row, one implementation

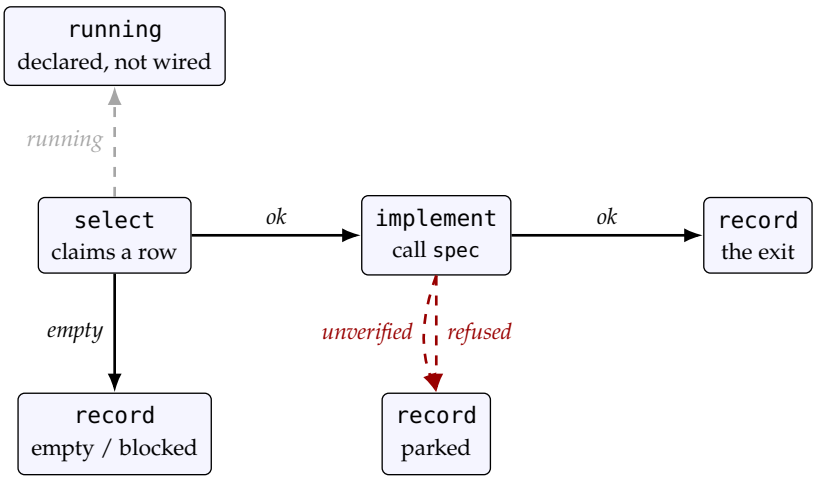


Figure 8.4: `queue` has three nodes: a claim, a call, a record. The grey port is declared by the node and has no edge, which is why a contested claim stops the run.

purpose	implement one queue row: claim it, call the spec graph with the row's spec, record the verdict
nodes	`select` select; `implement` call(`spec`, pinned); `record` record
ports	`select.ok`, `select.empty`, `select.blocked`; `implement.ok`, `.unverified`, `.refused`; `record.ok`
entry/exit, caps	`select` → `record`; `max_wall_ms = 6000000`; spawn, proc, fs_read, fs_write
when	an operator or `night` has rows to work; run input carries `workdir` and `base`
fails	a `running` claim (another run holds the row) has no edge, so the run ends without recording; `empty` and `blocked` reach `record` and write nothing; a refusal is recorded as `parked`, not as a failure

`record` writes to the queue store and to memory: the item's status becomes `done` when the arriving value is the literal `done` or a verdict of `passed`, and `parked` with the failing criterion otherwise. It also writes one memory document per row, which is how a later digest knows what was implemented.

`tdd` — a loop with three ways to prove nothing

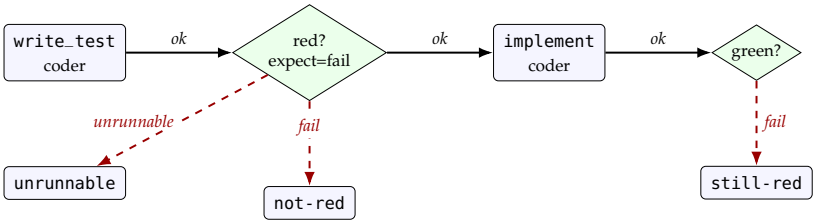


Figure 8.5: The RED proof is the whole point: a test that passes before the code exists leaves by `not-red`, and a criterion that cannot run leaves by `unrunnable` rather than counting as a red.

purpose	one test-driven iteration: write the tests, prove them red, implement, prove them green
nodes	`write_test` coder; `red` judge(expect fail); `implement` coder; `green` judge; `done`; and four `refuse` terminals
ports	`ok/err` on the coders; `ok/fail/unrunnable` on `red`; `ok/fail` on `green`
entry/exit	`write_test` → `green`; `max_wall_ms = 4200000`; coders at `timeout_ms = 1800000`
caps	spawn, proc, fs_read, fs_write
when	a spec whose exit criteria name behaviours; it is the loop `sdd` calls
lint	exit 3: `not-red`, `unrunnable` and `still-red` each declare `join = "any"` with one inbound edge (V9), so the graph does not load
fails	`not-red` (tests pass already), `unrunnable` (a criterion cannot run), `still-red` (the code did not make it pass), `refused` (the agent failed)

Only one of the four terminals (`still-red`) means the code was wrong. `not-red` means the test was worthless — a defect in the spec or in the test, not in the implementation.

`sdd` — the process around the loop

purpose	implement one spec under a contract: prove the criteria fail first, run `tdd`, verify, document, re-verify
nodes	`contract` judge(expect fail); `tdd` call; `verify` judge; `docs` call; `reverify` judge; `done`; `unverified`, `refused`
ports	`contract.ok/fail/unrunnable`; `tdd.ok/fail`; `verify.ok/fail`; `docs.ok/err`; `reverify.ok/fail`
entry/exit	`contract` → `done`; needs `agent`, `model`, `provider`, `spec`, `workdir`, `base`, `docs_brief`
caps	spawn, proc, fs_read, fs_write
when	you have a spec and want the whole cycle, including documentation, in one run
lint	exit 3: the spliced `tdd` nodes fail V9 under the `tdd__` prefix
fails	contract `fail` means the criteria already pass, which is a spec that proves nothing; docs that break the criteria end at `unverified`; a refusal is not returned to a host

`spec` — one spec, by hand

purpose	implement one spec and document it, with three honest verdicts
nodes	`code` coder; `verify` judge; `docs` call; `reverify` judge; `done`; `unverified`; `refused`
ports	`code.ok/err`; `verify.ok/fail`; `docs.ok/err`; `reverify.ok/fail`
entry/exit	`code` → `done`; `max_micro_usd = 1000000`, `max_wall_ms = 6000000`
caps	spawn, proc, fs_read, fs_write
when	you have a spec and want code plus docs without the TDD contract step
fails	`unverified` is the honest middle: code exists and the criteria fail; `refused` means the coder or the docs call failed

`docs` and `doc-code` — documentation, called and checked

purpose	write documentation for whatever brief the host passes; no inventory, no criteria, no opinion about truth
nodes	`docs` coder (the entry and the exit)
ports	`ok` on success; the coder's `err` otherwise
entry/exit	`docs` → `docs`; needs `agent`, `model`, `provider`, `spec`, `workdir`, `brief`; `timeout_ms = 1200000`
caps	spawn, proc
when	never directly: it is called by `spec`, `sdd` and `doc-code`
fails	called without a `brief` it has nothing to write; it cannot tell whether the document is true, which is why `doc-code` exists

purpose	document the code itself: list the modules with no `//!` header, write for them, then prove every path and symbol the document names exists
nodes	`inventory` tool; `select` route; `nothing-to-do` transform; `write` call(`docs`); `verify` tool; `gate` route; `done`; `unverified`
ports	`select.nothing-to-do/write`; `gate.pass/fail`; `ok/err` on the tools and the call
entry/exit, caps	`inventory` → `done`; `max_wall_ms = 1800000`; spawn, proc, fs_read, fs_write
when	a tree with undocumented modules; a documented tree ends at `nothing-to-do` and writes nothing
fails	an inventory that cannot run, a docs call that fails, or a document naming a path that does not exist all end at `unverified`

`self-improve` — a finding becomes work

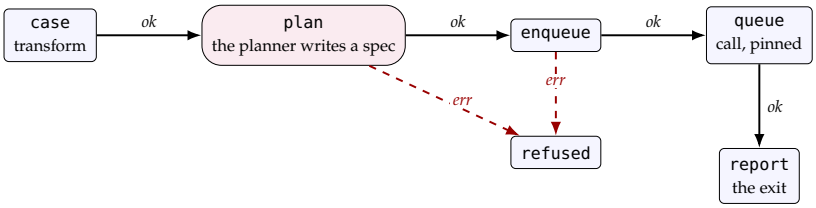


Figure 8.6: A finding becomes one spec, the spec becomes one row, and the queue implements it. The planner and the implementer are different models on purpose.

purpose	turn a finding into a spec, admit it to the queue, and run the row
nodes	`case` transform; `plan` plan(model `glm-5.3`); `enqueue`; `queue` call(pinned); `report` transform; `refused` transform
ports	`ok`/`err` on `plan` and `enqueue`; `queue.ok` only is wired
entry/exit, caps	`case` → `report`; `max_wall_ms = 6300000`; spawn, proc, fs_read, fs_write
when	you have a finding and no spec; the input carries `finding`, `workdir`, `base`
fails	a spec that fails the house shape is refused at `enqueue`; the queue's `unverified`/`refused` ports are not wired, so a failed implementation stops the run at the call; the queue claims the next pending row, which is the new one only when nothing older waits

8.6 The coder samples

These four exist to prove one thing each, and none of them is a production lane.

purpose	probe a Claude login through pii in one hard-capped turn; the verdict is the exit code
nodes	`code` coder(agent `claude`, model `haiku`); `done`; `refused`
ports	`ok` → `done`; `err` → `refused`
entry/exit	`code` → —; `max_budget_usd = 0.30`, `timeout_ms = 150000`
caps	spawn, proc
when	before trusting a lane that uses Claude; it costs one turn
fails	a missing login file (the credential arrives through `HOME`, never the graph); the scratch workdir must exist; 150 seconds is the whole budget

purpose	the smallest working coder: one coder node, one verdict
nodes	`code` coder(agent `pi`, model `deepseek-v4-flash`); `done`; `refused`
ports	`code.ok/err`
entry/exit	`code` → —; `max_wall_ms = 2100000`, `timeout_ms = 1800000`
caps	spawn, proc
when	as a template for a new coder graph
fails	the model id is dead: no provider serves `deepseek-v4-flash`, so the run dies at spawn; the workdir is a literal path, so it does not travel

purpose	the same coder inside a container, with `/tmp` as its workdir
nodes	`code` coder(agent `pi`, model `deepseek-v4-flash`); `done`; `refused`
ports	`code.ok/err`
entry/exit, caps	`code` → —; `max_wall_ms = 2100000`; caps: spawn, proc
when	as a template for the podman lane; the fragment it was meant to attach cannot be woven into it
fails	the same dead model id; a `/tmp` workdir outside the run's write scope is refused by the sandbox rather than silently ignored

purpose	a coder that fixes its own spec, on the current model
nodes	`code` coder(agent `pi`, model `deepseek-v4.1-flash:cloud`); `done`; `refused`
ports	`code.ok/err`
entry/exit,	`code` → —; `max_wall_ms = 2100000`;
caps	caps: spawn, proc
when	a self-fix lane that must run without a human writing the spec
fails	it once declared `name = "coder-demo"` and collided with that graph in the installed directory; the name is now its own, and a collision is the failure to check after copying a graph

8.7 The awareness graphs

purpose	build the digest on demand: two recalls and a post, with the timer removed
nodes	`devices` recall; `lessons` recall; `digest` post; `refused_devices`, `refused_lessons` transforms
ports	`ok/err` on both recalls
entry/exit	`devices` → —; caps memory_read, post
when	a person asks for the digest now; the scheduled version cannot answer, because its entry is a timer and a run of it reports "suspended, in_ms ≈ N"
fails	a recall that errors ends at its own refusal terminal; a missing `to` in the input fails at the post

purpose	the scheduled digest: recall devices, recall lessons, post into a thread
nodes	`tick` timer; `devices`; `lessons`; `digest` post; two refusal terminals
ports	`tick.fired`; `ok/err` on the recalls
entry/exit	`tick` → —; caps memory_read, post; `limit = 50`
when	a channel wants a recurring digest; the input carries the thread id
fails	asking it to run "now" suspends it by construction; a channel that is not configured fails at the post

purpose	triage one inbound Discord message with rules and no model
nodes	`classify`; `reply` post; `done`
ports	`classify.ok` → `reply`; `reply.ok` → `done`
entry/exit	`classify` → —; caps post only
when	a channel ingress starts it per message; input carries text, chat, user, channel
fails	an unrecognised text is classified `chat`; a reply goes to `$.input.chat`, so a message without a chat id cannot be answered

purpose	`code-index`: index the repository into memory on a schedule. `intervention-report`: turn an intervention's events into a dated report, gate it, post it, remember it
nodes	`code-index`: `start` timer, `index` worker, `gate`, `record` memory. `intervention-report`: `start` timer, `evidence` memory, `write` worker, `check` gate, `post`, `record` memory
ports	none readable: both express edges as `depends_on` node fields, which this loader does not read
entry/exit,	none; both put `name` in a `[graph]` table, so neither loads;
caps	caps: none declared
when	never in this build: `pii graph lint` refuses both with `invalid: missing 'name'` (exit 2)
fails	at load, before anything runs. `code-index`'s own header records why it is not ready: the index verb it needs was a script dependency the production rule forbids

8.8 Preflight and the fragment

purpose	prove the exit criteria are runnable before any agent is spawned
nodes	`verify` judge; `done`; `refuse`
ports	`verify.ok` → `done`; `verify.fail` → `refuse`
entry/exit	`verify` → —; caps proc only; `max_wall_ms = 600000`
when	the lane runs it through the daemon before the real run; the judge's sandbox is the real judge's, so a criterion that cannot write the scratch refuses here
fails	a criterion that cannot run ends at `refuse`, having spent no agent and no model budget

purpose	a blocking guard: refuse a run whose input asks for podman, pass every other run
nodes	`check` route; `refuse` refuse
ports	`check.refuse` → `refuse`; every other input falls through
entry/exit, caps	none: it is a fragment, attached by name with a sha256 pin; caps: none of its own; it inherits the host's
when	attach it to a host at a pre surface (`run.start` or `node.before`), which is allowed because it may block
fails	attaching it by name without a pin installs whatever the file says today; the weave refuses an unwoven attach rather than running the host without its guard

8.9 The starter graphs

Twelve samples under `graphs/starter/`: templates and test seams, not production lanes. Two exist only to break something on purpose.

Graph	Nodes and ports	Entry, caps	When, and how it fails
`assistant`	`agent`; `reply`; `fail_err`; `fail_budget`. Ports `ok`, `err`, `budget`	`agent`; fs_read, fs_write, proc, net	an assistant loop with a turn cap (4) and a wall of 600000 ms. A budget exit is a separate terminal from an error.
`boss-watch`	`tick`; `check` heal_detect; `cyc`; `escalate` gate; `done`. Ports `ok`, `flake`, `inconclusive`, `detected`	`tick`; proc	watches a boss process every 30 minutes. A detected failure suspends at the gate until a human decides; there is no `fs_write` anywhere in it.
`consolidate`	`tick`; `run` memory; `done`. Port `fired`, `ok`	`tick`; none	a scheduled consolidation. It has no error edge, so a failing memory node ends the run rather than routing it.
`crash`	`boom` debug_crash. No ports	`boom`; none	deliberately crashes the daemon to exercise the panic hook. Never install it on a host you care about.
`failover`	`primary`, `secondary` agents; two routes; retry and compress transforms; `exhausted`; `done`	`primary`; fs_read, fs_write, proc	a model-failover ladder. `compress` and `exhausted` are deliberate dead ends: retrying cannot fix a payload that is too large.
`heal-fix`	`probe`; `route_probe`; `unexpected_ok`; `fix` agent; `retest`; `route_retest`; `retry`; `summarize`; `record_fixed`; `escalate`; `done`	`probe`; fs_read, fs_write, proc	localise, patch, validate, escalate, in a disposable worktree. The fix agent has file tools only, no shell; `bash` runs only in `tool` nodes.

Graph	Nodes and ports	Entry, caps	When, and how it fails
`heal-watch`	`tick`; `check` heal_detect; `escalate` gate; `done`. Ports `ok`, `flake`, `detected`	`tick`; proc	runs the quality gate every 30 minutes, writes one ledger row for a reproducible failure, and suspends for acknowledgement. Read-only detection.
`react`	`agent`; `done`; `fail`	`agent`; fs_read, fs_write, proc	react to one signal with 24 turns and compaction at 24k tokens. Both `err` and `budget` end at `fail`.
`slow`	`wait` timer; `done`	`wait`; none	a 30-second wait, used to have a run in flight at SIGTERM. Not a workload.
`soak`	`step` transform	`step`; none	one soak step, bounded to 100 steps and 30 seconds. The harness runs many of them.
`soak_cron`	`tick`; `done`	`tick`; none	the same soak on a timer, bounded to 600000 ms.
`verify-fix`	`fix` agent; `test` tool; `gate` route; `retry`; `done`. Ports `pass`, `fail`	`fix`; fs_read, fs_write, proc	fix then verify, looping at most 3 times. A `fail` that keeps failing is a run that ends at the budget, not at a verdict.

8.10 What the graphs can do to the machine

The ladder is checked at load time: this is the set of refusals an author will meet.

Capability	What it allows, and the refusal when it is missing
`fs_read`	read the working tree
`fs_write`	modify the working tree. A coder without it stops and reports that no write tool is available; measured 2026-11-03, it did not degrade silently.
`proc`	run a command. This is what lets a judge execute exit criteria, and what a `judge` node requires.
`spawn`	start an agent process; every model call needs it
`post`	send a message to a human; `night` and the report graphs hold it
`memory_read`	read the memory store; the digest graphs hold it
`net`	reach the network under `[net] grants`; only the assistant sample holds it

Two refusals are worth being able to read at sight. `graph.call.widen` means a call needs a capability its host does not hold — the fix is in the host's `caps`, not in the callee. And the lint code `V2` means a node needs a capability the graph does not declare; the message names both the node and the capability, and it is the most common error in a new graph.

8.11 Reading a graph's failure

A run that ends *without* reaching a terminal node has not failed loudly: it took a port with no edge, or it was spliced into a host that does not wire its refusals. The trace answers that in one query — `--terminal` names where it stopped, `--reason` why.

```
pii graph lint graphs/night.toml        # V-codes, pins, caps before anything runs
pii graph list                          # installed vs repo: DRIFT is the interesting column
pii graph explain queue                 # nodes and ports in execution order
pii run start queue --input '{"workdir":"/projects/pi-improved","base":"<40-hex>"}'
pii run trace "$RUN" --terminal         # where it stopped
pii run trace "$RUN" --reason implement # why that node ended there
```

9 Memory

Everything else in this manual is about work: runs, rows, graphs, gates. This chapter is about what survives the work — what pii remembers after a run ends, how it is found again, why some of it fades, and what an operator should check when memory looks wrong. Scattered in earlier chapters you will have seen memory mentioned next to the stores; that was a reference. This is the chapter.

The short version, before the detail: **pii does not remember conversations. It remembers documents.** A document is a piece of text with a layer, a kind, a set of concepts, and an id derived from its own content. Nothing is remembered by being said; something is remembered by being written, and writing the same text twice is the same document.

9.1 The four things a document carries

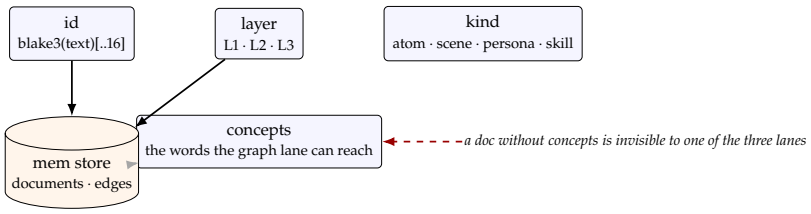


Figure 9.1: The four fields that matter operationally. The id is derived, not assigned: two puts of the same text produce one document.

Because the id is the hash of the text, `pii memory put` is *idempotent*: writing the same sentence twice does not create two memories, it rewrites one. That is also why editing a memory means writing a new document — the text changed, so the id changed — and why a correction appears as a new document rather than a modification of the old one.

9.2 The three layers, and what each is for

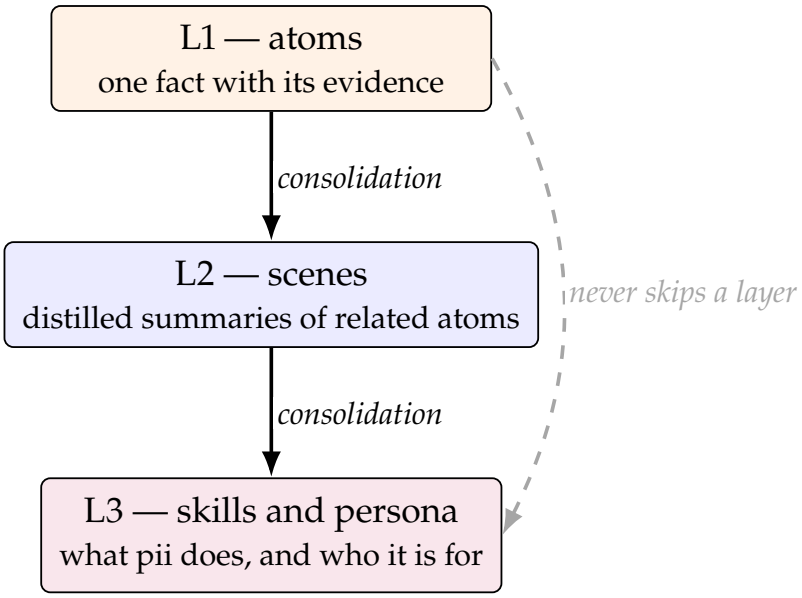


Figure 9.2: Consolidation moves upward and never backward. An atom is not a scene that lost detail; a scene is a set of atoms that agreed.

The layers are not storage tiers — they are kinds of claim. An **atom** is one fact with evidence: a single observation it would be wrong to generalise from. A **scene** is a distillation of several atoms that share concepts: it says what the atoms together mean. **L3** holds skills — procedures pii can follow — and the persona document, which is the one kind that never decays.

Two consequences an operator should hold on to. First, a scene is derived: if you delete every atom behind it, the scene remains but its evidence is gone, which is a reason to archive rather than delete. Second, `same_layer_only` is on: consolidation will not merge an atom into a scene, so a half-finished idea stays an atom until it earns its way up.

9.3 How a memory is found: three lanes, one ranked list

A search does not consult one index. Three independent lanes each produce candidates, and their rankings are fused.

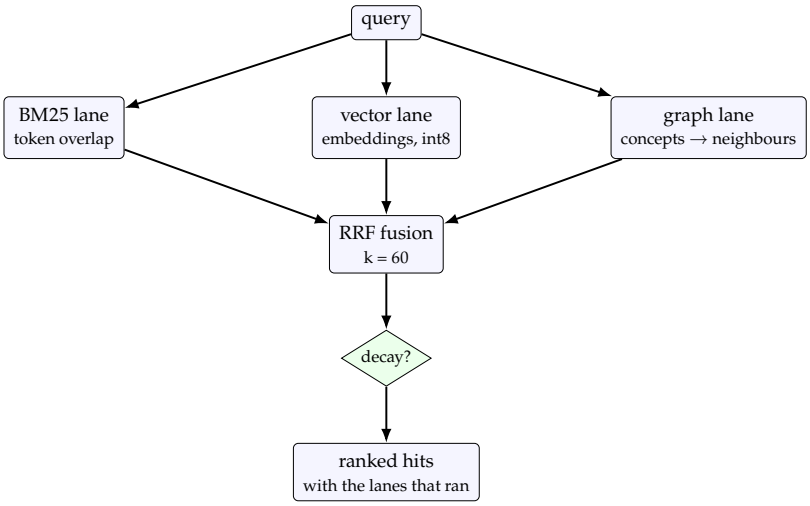


Figure 9.3: Fusion by rank, not by score. Reciprocal-rank fusion is used because the three lanes produce scores on incomparable scales — BM25 counts terms, cosine sits in $[-1, 1]$, the graph lane counts hops.

Reciprocal-rank fusion with $k = 60$ combines them: each lane contributes $1/(k + \text{rank})$ per hit, and the sums decide the order. The practical effect is that a document ranked first by one lane beats a document ranked fifth by all three, which is the behaviour you want when one lane is much more confident than the rest.

Then decay is applied per kind, and here is the first thing that surprises operators:

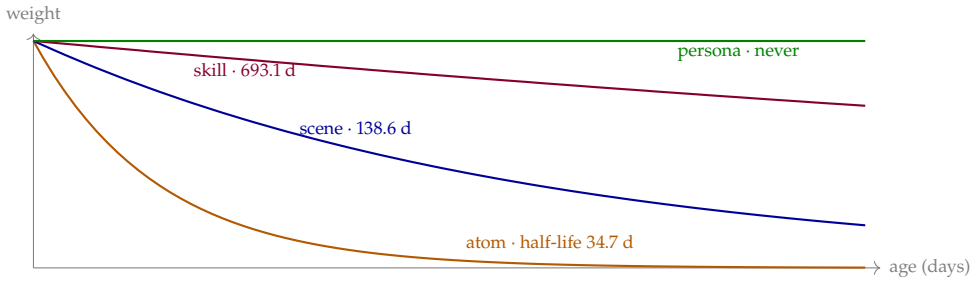


Figure 9.4: Decay is exponential, score $\times\, e^{-\lambda\,\text{age}}$, with λ fixed per kind. The half-lives are computed from the constants in the crate: 0.693/0.02, 0.693/0.005, 0.693/0.001.

An atom written a month ago carries roughly half the weight of one written today; a skill written two years ago still carries most of its weight. Decay reorders results, it does not delete anything — deletion is retention's job, and it is a separate mechanism (chapter 4).

9.4 The graph lane, and why it often contributes nothing

This is the intricacy most worth knowing, because it produces the most confusing symptom: a search that returns obviously-related documents

from BM25 and from vectors, while the graph lane appears in the response and adds nothing.

The graph lane fires only when *a query token is already a concept head present in the edge table* — that is, when some earlier document was written with that word in its concept list. Free-text queries against documents stored without explicit concepts produce no query concepts at all, so the lane contributes nothing.

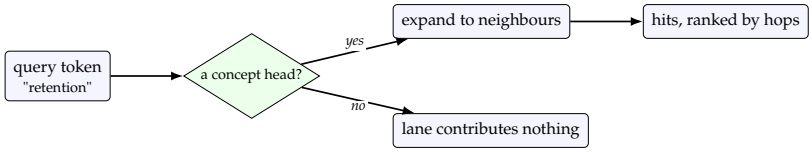


Figure 9.5: The lane reports that it RAN either way. "Ran" is not "hit" — the response lists the lanes that were consulted, not the lanes that found something.

The operational reading: if the graph lane never contributes on your store, your documents were written without concepts. `pii memory put` takes the text; the concept list is what makes a document reachable by association. Writing facts through the fact path records concepts, and that is how a store becomes navigable rather than merely searchable.

9.5 Consolidation: when many atoms become one scene

Consolidation is the process that turns atoms into a scene. It is deliberately conservative, and each of its limits is worth knowing because each one explains a scene that never appeared.

Rule	Value	What it means to an operator
shared concepts	at least 2	a single shared word is never enough to merge
concept Jaccard	at least 0.15	the two documents must be about mostly the same thing
negation window	8 tokens	a "not" near a shared concept vetoes the merge
candidate cap	200	generation stops at 200 and says so in the report
same layer only	on	an atom is never merged into a scene
archive after	configured days	documents past their last use are archived, not deleted

The negation window is the one that saves you: "the disk floor is 12 GB" and "the disk floor is not 12 GB" share every concept, and would otherwise consolidate into a scene that asserts both.

9.6 The commands, and what each one touches

Command	What it does	Reads / writes
`memory search`	fused search over the three lanes, returns hits and the lanes that ran	reads documents and edges
`memory put`	writes a document; the id is the content hash, so a repeat is a no-op	writes one document
`memory learn`	records an outcome as a document	writes one document
`memory tiers`	what sits in L1, L2 and L3	reads only
`memory entity`	creates or reads an entity in the entity graph	writes entities
`memory link`	creates an edge between entities	writes edges
`memory neighbours`	the entities adjacent to one entity	reads edges
`memory consolidate`	generates candidate merges under the rules above	writes scenes, archives
`memory compact`	removes superseded versions of a document	writes
`memory show`	one document, in full, with its metadata	reads
`memory archive` / `restore`	takes a document out of search, or puts it back	writes

Most of these go through the daemon, because the daemon is the only writer. If you are told `uds.connect.failed`, the daemon is down — that is a supervision problem (chapter 2), not a memory problem, and it is worth saying plainly because the error names a socket rather than the cause.

9.7 What goes wrong, in the order you will meet it

Symptom	Cause	What to do
results are stale but present	decay, not deletion	search again with a recency bias, or re-put the fact
a fact is gone	retention swept it, or it was archived	`memory show` the id; restore from backup if needed
the graph lane never contributes	documents were written without concepts	write facts through the fact path
the vector lane is skipped	no embedder configured, or zero dimensions	the response marks itself degraded; BM25 and the graph still answer
two documents say the same thing	the text differed by a character	the ids differ by design; consolidate or archive

Symptom	Cause	What to do
a scene asserts something odd	merged atoms that disagreed	the negation window covers only 8 tokens; tighten the concepts
search is slow	a large store with no compact	run `memory compact`, then measure again

9.8 How memory is bounded

Memory is bounded in four independent ways, and an administrator should know which one is doing the work on their host:

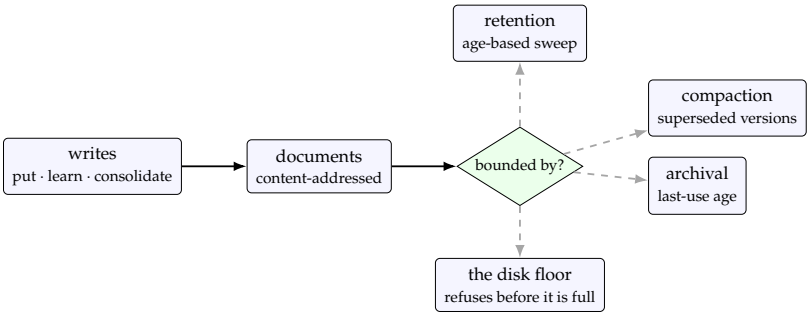


Figure 9.6: Four bounds, four mechanisms, four different failure modes. "Memory is full" is never one thing; the question is which bound fired.

The order to investigate: `pii store df` first (how big it actually is), then retention (chapter 4) for what age has swept, then compaction for how much is superseded copies, then archival for what consolidation moved aside. Only then is it a capacity question rather than a policy one.

Where to look next

This manual deliberately stops at the boundary of behaviour. When a symptom does not match what is written here, the two places that answer faster than any guess are:

`pii health` what the daemon believes about itself right now, with the reasons attached rather than a colour.
`pii run trace <id>` the journal of one run, node by node, which is the only account of what actually happened.

The architecture manual explains why those two answers are trustworthy.